

國立陽明交通大學
資訊科學與工程研究所
碩士論文

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University
Master Thesis

基於因果圖的多源日誌關聯結合大型語言模型驅動之攻擊
生命週期補全
LLM-driven Attack Lifecycle Completion using Causal
Graph-based Correlation from Multi-source Logs

研 究 生：蘇敬堯 (Su, Ching-Yao)
指導教授：林盈達 (Lin, Ying-Dar)

中華民國 一一五年六月
June 2026

基於因果圖的多源日誌關聯結合大型語言模型驅動
之攻擊生命週期補全

LLM-driven Attack Lifecycle Completion using Causal
Graph-based Correlation from Multi-source Logs

研 究 生：蘇敬堯

Student：Ching-Yao Su

指導教授：林盈達 博士

Advisor：Dr. Ying-Dar Lin

國立陽明交通大學

資訊科學與工程研究所

碩士論文

A Thesis

Submitted to Institute of Computer Science and Engineering

College of Computer Science

National Yang Ming Chiao Tung University

in partial Fulfillment of the Requirements

for the Degree of

Master of Science

in

Computer Science

June 2026

Taiwan, Republic of China

中華民國 一一五年六月

誌 謝

謝謝

蘇敬堯 於

國立陽明交通大學 資訊科學與工程研究所

中華民國 一一五年六月



基於因果圖的多源日誌關聯結合大型語言模型驅動之攻擊生命週期補全

學生：蘇敬堯

指導教授：林盈達 博士

國立陽明交通大學
資訊科學與工程研究所

摘 要

中文摘要

關鍵字：中文, 摘要, 關鍵詞, 5-7 個, 不要多, 也不要少



LLM-driven Attack Lifecycle Completion using Causal Graph-based Correlation from Multi-source Logs

Student : Ching-Yao Su

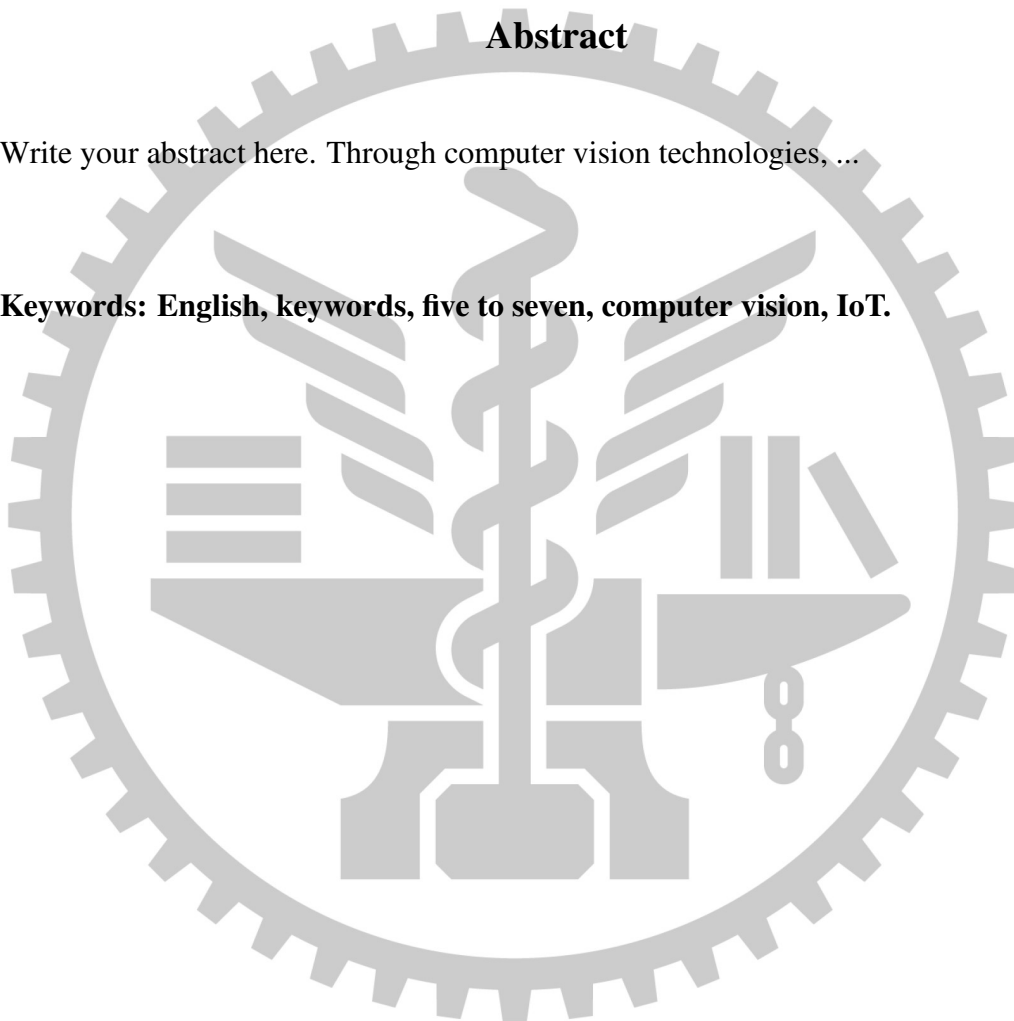
Advisor: Dr. Ying-Dar Lin

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University

Abstract

Write your abstract here. Through computer vision technologies, ...

Keywords: English, keywords, five to seven, computer vision, IoT.



Contents

Chinese Abstract	i
English Abstract	ii
Contents	iii
List of Figures	vi
List of Tables	vii
1 Introduction	1
2 Background and Related Works	5
2.1 MITRE ATT&CK Framework and Attack Lifecycle	5
2.2 Security Monitoring and Logging	7
2.2.1 Endpoint Detection and Response (EDR)	7
2.2.2 Network Detection and Response (NDR)	8
2.2.3 Endpoint Logging	8
2.2.4 Network Logging	9
2.2.5 Example Alerts and Logs	9
2.3 Causal Graph for Log Correlation	10
2.4 Large Language Model	12

2.5	Related Works	13
3	Problem Formulation	16
3.1	Notation.....	16
3.2	Problem Statement.....	16
4	Solution Approach	20
4.1	Solution Overview	20
4.2	Graph-based Log Extraction.....	21
4.2.1	Log2Graph	22
4.2.2	Attack Subgraph Search.....	23
4.2.3	Graph2Log	26
4.3	LLM-Driven Lifecycle Completion.....	27
4.4	Case Study	29
5	Implementation	32
5.1	Experimental Environment.....	32
5.2	Data Collection	33
5.2.1	Attack Scenarios and Log Collection	33
5.2.2	Partial Lifecycle Derivation.....	34
5.3	Stage 1: Graph-based Log Extraction.....	35
5.4	Stage 2: LLM-Driven Lifecycle Completion.....	36
5.4.1	Model Selection and Fine-Tuning.....	36
5.4.2	Inference Pipeline	37

6	Evaluation and Results.....	38
6.1	Experimental Setup.....	38
6.2	Data Profiling.....	40
6.3	Lifecycle Completion: Finetuned LLM vs. Baseline Models	42
6.4	Factor Contribution.....	43
6.4.1	Graph-based Log Extraction: Clean Input vs. Noisy Input	43
6.4.2	LLM Fine-tuning: Finetuned vs. Non-Finetuned.....	46
6.4.3	Time-Window Size: $W=5$ vs. Alternative Sizes	47
6.5	Behavioral Analysis.....	48
6.5.1	Sample Complexity: Single-Technique vs. Multi-Technique Samples	49
6.5.2	Alert-Gap Complexity: Sparse vs. Dense Alert-Gaps.....	51
6.5.3	Lifecycle Scale: Short vs. Long Lifecycles.....	52
7	Conclusion and Future Work	54
	References.....	56

List of Figures

Figure 1	Attack lifecycle of an example scenario.	6
Figure 2	Representative records from the four data sources.	10
Figure 3	A causal graph fragment for an attack scenario.	11
Figure 4	Schema of the problem formulation.	18
Figure 5	Solution overview.	21
Figure 6	Alert-gap partitioning of the example scenario.	31
Figure 7	Log and token reduction by Stage 1.	44
Figure 8	F1 distribution of Stage 1 extraction.	44
Figure 9	Lifecycle LCS-F1 vs. benign-log noise ratio.	45
Figure 10	Lifecycle LCS-F1 vs. window size W	47
Figure 11	Per alert-gap LCS-F1 vs. window size W	48
Figure 12	Overall per-window technique detection performance.	49
Figure 13	Per-window technique detection stratified by techniques per sample.	50
Figure 14	Per alert-gap LCS-F1 vs. techniques per alert-gap.	51
Figure 15	Lifecycle LCS-F1 vs. lifecycle length.	52

List of Tables

Table 1	Comparison of related works on attack detection and lifecycle reconstruction.	13
Table 2	Summary of notations.	17
Table 3	Causal graph node types and edge types.	23
Table 4	Open-source tools and libraries.	33
Table 5	Baseline and proposed-model configurations used throughout the evaluation.	39
Table 6	Lifecycle LCS-F1 comparison across models.	42
Table 7	Ablation of LLM fine-tuning.	46

Chapter 1.

Introduction

In modern cybersecurity operations, understanding the complete attack lifecycle is essential for effective defense. An attack lifecycle is the ordered sequence of MITRE ATT&CK techniques employed by adversaries during an intrusion [1]. By reconstructing this sequence, Security Operations Center (SOC) analysts can identify the attacker’s strategic goals, assess the full scope of compromise, and refine their defensive posture. However, current detection relies predominantly on alerts from Endpoint Detection and Response (EDR) and Network Detection and Response (NDR) tools [1, 2]. Because these rule-based engines require continuous maintenance and often fail to detect novel attack variants, alert-derived lifecycles miss approximately 60% of the techniques actually employed [1], leaving significant gaps in the attack narrative.

System and network logs offer a complementary evidence source, recording host and network activities continuously regardless of whether alerts are generated [3]. These logs can potentially fill the gaps left by partial alert-based lifecycles, but leveraging them for automated lifecycle completion poses two fundamental challenges. The *log imbalance problem*: attack-relevant logs account for less than 1% of all system logs, and this extreme ratio leads to poor detection when models analyze raw logs directly [4]. The *semantic gap problem*: MITRE ATT&CK techniques describe high-level adversary intent that traditional machine learning models cannot effectively classify from low-level log entries [5]. Prior work has addressed these challenges only partially — operating on alerts rather than raw logs, producing tactic-level

rather than technique-level output, or relying on hand-crafted features that do not generalize beyond the training distribution (Chapter 2).

Two recent advances directly motivate the design of this thesis. *Causal graphs* — directed structures that preserve causal and temporal dependencies between system entities and events [2, 4, 6] — unify isolated log entries into a relational map of information flow. Because attack activities form connected chains through shared processes, files, and network endpoints, causal-graph traversal provides a principled way to separate attack-relevant logs from surrounding benign noise. *Large Language Models (LLMs)*, in contrast, address the semantic side: pre-trained on vast corpora that include security documentation, they reason about log evidence in natural language and can be fine-tuned to map low-level log entries to high-level ATT&CK techniques [7, 8, 9]. Together, these capabilities motivate a decoupled design in which structural filtering is handled by graph algorithms and semantic classification is delegated to an fine-tuned LLM.

Building on these two capabilities, this thesis proposes a two-stage framework. Stage 1, Graph-based Log Extraction, transforms multi-source raw logs from endpoint loggers and network loggers into a causal graph, identifies the attack subgraph through alert-guided node scoring and backward tracing, and maps the subgraph back to a focused subset of attack-relevant logs, making downstream LLM analysis tractable. Stage 2, LLM-Driven Lifecycle Completion, partitions the extracted logs into fixed-length time windows and uses a reasoning-augmented fine-tuned LLM to classify each segment against ATT&CK technique definitions. The per-segment predictions are then assembled into a completed lifecycle through deduplication and interleaving with the alert-detected techniques.

To verify the effectiveness of this framework, experiments are conducted on 237 attack scenarios drawn from a public attack-emulation framework, covering 34 MITRE ATT&CK

techniques across 10 tactics. The proposed framework is compared against four representative baselines spanning classical machine learning (Decision Tree), sequence-aware deep learning (Bi-LSTM), and Non-Finetuned large-scale LLMs (GPT-5 and Qwen3.5-122B-A10B). Beyond the headline comparison, three factor analyses isolate the contribution of Stage 1 input quality, fine-tuning, and time-window size; three behavioral analyses examine performance under varying sample complexity, alert-gap density, and lifecycle length. The results show that the proposed framework substantially outperforms all baselines, that Stage 1 is a structural prerequisite without which the entire pipeline collapses, and that domain-specific fine-tuning rather than raw model scale drives the performance gain.

The main contributions of this thesis are:

- A causal graph-based log extraction framework that systematically isolates attack-relevant logs from the overwhelming volume of benign system noise, making downstream LLM analysis tractable.
- A reasoning-augmented fine-tuned LLM that detects MITRE ATT&CK techniques from extracted log evidence and assembles per-segment predictions into a completed attack lifecycle, bridging the semantic gap between low-level logs and high-level adversary intent.
- A systematic three-layer evaluation — headline comparison, factor contribution, and behavioral robustness — demonstrating that the proposed framework substantially outperforms both traditional ML and Non-Finetuned LLM baselines, and identifying which design decisions drive the gain.

The remainder of this thesis is organized as follows. Chapter 2 provides background on the MITRE ATT&CK framework, causal graphs, and Large Language Models, followed by a

review of related work. Chapter 3 formally defines the research problem. Chapter 4 details the proposed two-stage framework. Chapter 5 describes the implementation and experimental testbed. Chapter 6 presents the evaluation results. Chapter 7 concludes the thesis and outlines future directions.



Chapter 2.

Background and Related Works

The complementary relationship between alerts and logs motivates the two core challenges addressed by our framework. First, because raw logs are overwhelmingly dominated by benign activities, an effective structural filtering mechanism is needed to extract attack-relevant logs from the noise—a challenge addressed by causal graph-based correlation, introduced in Section 2.3. Second, because extracted logs carry no technique labels, a model capable of semantic reasoning is needed to determine which ATT&CK techniques the log evidence represents—a challenge addressed by Large Language Models, introduced in Section 2.4.

2.1 MITRE ATT&CK Framework and Attack Lifecycle

The MITRE ATT&CK framework is a public knowledge base that catalogs the behaviors used by adversaries in real-world cyber intrusions [10]. Unlike vendor-specific threat catalogs that differ from one another, ATT&CK provides a single shared vocabulary for describing every stage of an attack, from initial entry to final impact. Since its public release, it has been widely adopted by security operations centers, threat-intelligence teams, and academic researchers as the standard way to describe cyber threats.

The framework describes adversary behavior along two dimensions. *Tactics* are the goals an adversary pursues at each stage of an attack — gaining initial access, executing code, getting

higher privilege, evading defenses, stealing data, and so on; the Enterprise matrix lists twelve such tactics. Each tactic contains multiple *techniques* that describe the concrete methods used to achieve it. For example, the Execution tactic includes T1059 (Command and Scripting Interpreter), whose sub-techniques T1059.001 (PowerShell) and T1059.003 (Windows Command Shell) are two common implementations.

An *attack lifecycle* is a time-ordered sequence of ATT&CK techniques observed during an intrusion. Each technique corresponds to one adversary action, and the order reflects how the attack unfolded. Figure 1 shows one such lifecycle, taken from the experimental dataset used later in this thesis (Section 5.2); Section 4.4 traces the same scenario through the proposed framework.

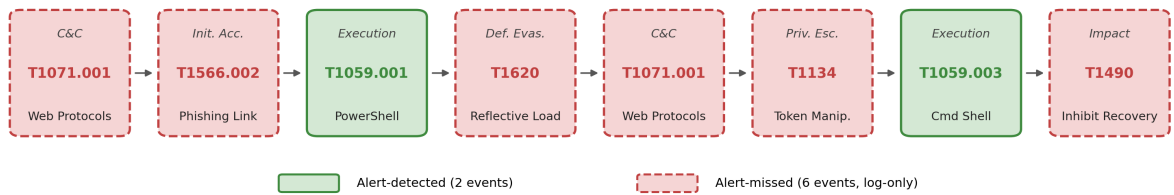


Figure 1: Attack lifecycle of an example scenario from the experimental dataset. Eight ATT&CK techniques progress from initial command-and-control to a final Impact step that disables system recovery. Green boxes are techniques caught by EDR or NDR alerts; red boxes are techniques the alerts miss — these events are recorded only in the underlying logs.

The scenario starts with an outbound command-and-control channel (T1071.001) and a spearphishing link (T1566.002) that tricks the victim into running a PowerShell script (T1059.001), which loads a payload DLL directly into memory (T1620). The loaded payload sends signals back over the same channel (T1071.001) and then injects code into a privileged Windows service to take over its access token (T1134). The attacker then opens a command shell (T1059.003) and disables the Windows System Restore scheduled task so that recovery is no longer possible (T1490), preparing for a destructive impact.

Only two of these eight events are caught by the rule-based alert stream, both from the Ex-

ecution tactic for which detection rules already exist. The other six — the two command-and-control channel events, the spearphishing link, the reflective DLL load, the access-token manipulation, and the System-Restore inhibition — do not trigger any alert, but they are recorded in full inside the host and network logs. This gap is what motivates the thesis: rebuilding the complete lifecycle requires looking into those logs and recovering the techniques that the rule sets missed.

2.2 Security Monitoring and Logging

Reconstructing an attack lifecycle requires evidence from both the endpoint and the network. Two complementary modalities cover these layers: rule-driven *alerts*, which are high-precision and already tagged with ATT&CK techniques but only fire on patterns encoded in the rule set, and continuously recorded *logs*, which capture every system or network event but carry no semantic label. Neither modality alone supports lifecycle reconstruction — alerts are accurate but sparse, while logs are exhaustive but unstructured — so the two are used together. The following subsections introduce the four resulting sources: EDR alerts (Section 2.2.1), NDR alerts (Section 2.2.2), endpoint logs (Section 2.2.3), and network logs (Section 2.2.4); representative records of each appear in Section 2.2.5.

2.2.1 Endpoint Detection and Response (EDR)

Endpoint Detection and Response (EDR) systems continuously monitor host-level activity — process execution, file operations, registry access, and similar events — and apply predefined detection rules to flag suspicious behaviour. Modern EDR platforms automatically map every alert to a specific MITRE ATT&CK technique. This eliminates the need for manual labeling later and immediately shows where the attack fits in the overall lifecycle. Alerts also carry metadata

that precisely identifies the host-level entities involved, such as process identifiers, file paths, and command lines.

This precision comes at the cost of coverage. Because EDR depends on a finite rule set, it cannot flag techniques that fall outside the encoded patterns and degrades on emerging variants that have not yet been catalogued. The resulting alert stream is high-precision but inherently sparse, providing only a fragmented view of the underlying lifecycle.

2.2.2 Network Detection and Response (NDR)

Network Detection and Response (NDR) systems play an complementary role at the network layer: they inspect traffic across all monitored hosts and apply signature-based or behavioural rules to flag suspicious flows. Like EDR, mature NDR engines emit alerts already tagged with ATT&CK techniques, and they capture behaviour invisible to host-based monitoring — application-layer protocol misuse (T1071), exfiltration over alternative channels (T1048), and similar network-only techniques.

NDR shares EDR's structural limitation. Both depend on rule sets that cover only a fraction of the technique universe, so even the union of EDR and NDR alerts still misses the techniques outside their rules. Techniques not encoded as rules leave traces only in the underlying logs, motivating the two logging sources described next.

2.2.3 Endpoint Logging

Endpoint logging records host-level activity continuously, independently of whether any detection rule fires. Typical endpoint loggers operate at kernel or system-service level and capture process creation, file operations, network connections, image loads, and similar low-level events, attaching to each event a stable identifier for the originating process so that execution

chains can be reconstructed through parent–child relationships.

The trade-off is that endpoint logs are unstructured with respect to attack semantics. Each entry records a single operation in isolation with no indication of whether it is benign or malicious, so the technique-level meaning of the activity must be recovered from the entries themselves. The volume is also large: only a small fraction of the recorded events ever relate to an attack.

2.2.4 Network Logging

Network logging provides the same kind of continuous, label-free record at the network layer, capturing flow metadata, DNS queries, HTTP transactions, TLS handshakes, and similar communication events that may not trigger any signature-based alert. Modern NDR engines typically record observed traffic continuously while applying detection rules to it in parallel; the log stream therefore covers every packet seen, not only those that triggered an alert.

As with endpoint logs, network logs are unstructured with respect to attack semantics: each entry describes a single communication event without indicating its role in an attack chain. They also do not record which host-level process initiated the traffic, so determining the originator requires correlation against the endpoint logs.

2.2.5 Example Alerts and Logs

Figure 2 shows one representative record from each of the four sources, all drawn from the same attack scenario. The 2×2 layout follows the two organising axes introduced above: the upper row (a, b) holds the two rule-driven alerts, the lower row (c, d) holds the corresponding raw log entries, the left column (a, c) is endpoint-side, and the right column (b, d) is network-side. The endpoint pair captures the same PowerShell invocation from two perspectives — (a)

as an EDR alert already tagged with technique T1059.001, and (c) as the underlying process-creation log entry recording the command line and parent process. The network pair captures the corresponding payload download — (b) as an NDR alert flagging a suspicious HTTP banner, and (d) as the raw HTTP GET that retrieved the loader. Together the four records illustrate the alert-versus-log contrast on both layers: the alerts arrive with technique labels but only for patterns the rules recognise, while the logs preserve every event but require downstream analysis to recover the same semantics.

(a) EDR alert

```
timestamp: 2026-04-19T18:51:14.901Z
rule.id: 92029
rule.desc: Powershell executed script from
           suspicious location
rule.mitre: T1059.001 (PowerShell, Execution)
event.eventdata:
  processGuid: {E20E79AD-2420-69E5-0000-00102FB41800}
  image: C:\Windows\System32\WindowsPowerShell
         \v1.0\powershell.exe
  commandLine: powershell.exe -ExecutionPolicy Bypass
               -File C:\Users\Administrator\loader.ps1
  parentImage: C:\Program Files\OpenSSH\bin\sh.exe
  user: VAGRANT-2008R2\Administrator
```

(b) NDR alert

```
timestamp: 2026-04-20T02:53:07.198Z
event_type: alert
src: 192.168.81.16:8000
dst: 192.168.81.11:49350 (TCP)
alert:
  signature: ET INFO Python SimpleHTTP
           ServerBanner
  sig_id: 2034636
  category: Misc activity
  severity: 3
```

(c) Endpoint log entry

```
TimeCreated: 2026-04-19T18:51:12.844Z
EventID: 1 (Process Create)
ProcessGuid: {E20E79AD-2420-69E5-0000-00102FB41800}
Image: C:\Windows\System32\WindowsPowerShell
       \v1.0\powershell.exe
CommandLine: powershell.exe -ExecutionPolicy Bypass
              -File C:\Users\Administrator\loader.ps1
ParentProcessGuid: {E20E79AD-2420-69E5-0000-00100DB31800}
ParentImage: C:\Program Files\OpenSSH\bin\sh.exe
User: VAGRANT-2008R2\Administrator
LogonGuid: {E20E79AD-23DC-69E5-0000-002082990100}
```

(d) Network log entry

```
timestamp: 2026-04-20T02:53:07.404Z
event_type: http
src: 192.168.81.11:49350
dst: 192.168.81.16:8000 (TCP)
http:
  method: GET
  url: /payload_2.exe
  user_agent: Mozilla/5.0 (Windows NT 6.1)
            WindowsPowerShell/5.0.10586.117
  protocol: HTTP/1.1
  status: 200
```

Figure 2: Representative records from each of the four data sources: (a) EDR alert with ATT&CK technique tag, (b) NDR alert, (c) endpoint log entry (process-creation event), and (d) network log entry (HTTP GET request).

2.3 Causal Graph for Log Correlation

A causal graph is a directed structure that preserves temporal and functional dependencies between system entities. Nodes represent the entities involved in system activity — processes, files, and network endpoints — and edges represent the causal events that link them: a process creating another process, a process writing or reading a file, a process opening a network con-

nection, and so on. By mapping each individual log entry, whether endpoint or network, to a directed edge between two such entities, the framework transforms the otherwise isolated log stream into a unified relational map of information flow across the monitored environment.

Figure 3 illustrates this structure on a small attack fragment. A malicious document, delivered through email, causes Outlook to spawn Word, which in turn launches PowerShell; the PowerShell process opens a network connection to a command-and-control endpoint, writes Loader.dll to disk, and uses Rundll32 to execute the loader. The loader then reads a sensitive file, runs reconnaissance commands (whoami, net.exe), and persists itself through a Registry Run key. Each of these actions is recorded by an individual log entry, and each appears in the graph as a directed edge between two entities. The red nodes and edges form the connected subgraph that constitutes the attack itself; the black branches in the same graph come from unrelated, benign activity that happens to share the same root process. The four ATT&CK techniques annotated on the right — T1059.001 (PowerShell), T1105 (Ingress Tool Transfer), T1033 (System Owner/User Discovery), and T1547.001 (Registry Run Keys) — map to specific edges or short paths within this subgraph.

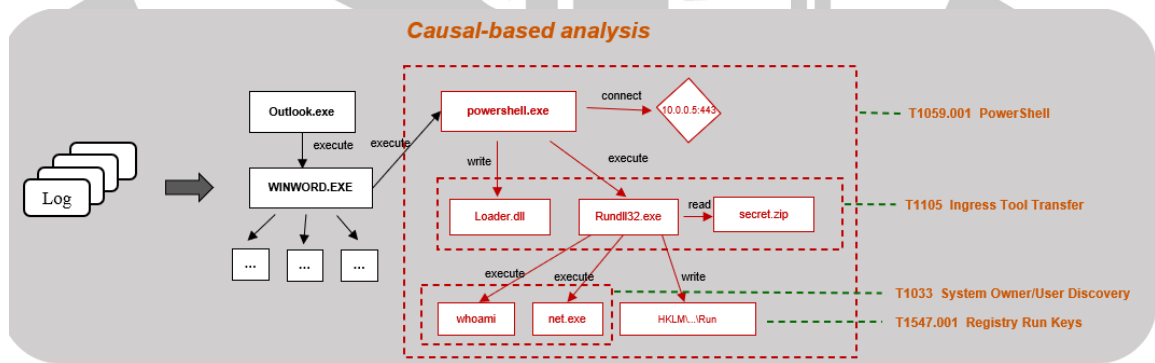


Figure 3: A causal graph fragment for a single attack scenario. Black nodes and edges denote benign activity; red nodes and edges form the connected attack subgraph. Multiple ATT&CK techniques manifest as edges or short paths within the same connected subgraph.

This representation directly enables the graph-theoretic search developed in Chapter 4. An attack lifecycle is, by construction, a chain of causally connected actions across a small number

of attacker-controlled entities, so it manifests in the graph as a connected subgraph rather than as a scattered set of records. Given any alert that flags a single node within this subgraph, the rest of the chain becomes reachable by simple traversal of the graph’s edges: backward along the causal links to find where the attack entered the system, forward to find its downstream impact. This is the operation that Stage 1 of the proposed framework (Section 4.2) realises in order to produce the attack-relevant log set $\hat{L}$.

2.4 Large Language Model

Even after structural filtering, the extracted logs carry no technique label. Mapping them to MITRE ATT&CK techniques requires a classifier that reasons about technique semantics rather than matching surface patterns, because ATT&CK techniques describe abstract adversary behaviour: a single technique may be implemented in many different ways and therefore manifest in many different log patterns.

Traditional machine-learning approaches (e.g., decision trees, LSTMs) rely on hand-crafted or surface features and generalise poorly to implementations they have not seen, since the technique abstraction is absent from the feature representation. Large Language Models (LLMs) start from a better position: Transformer self-attention reasons jointly over all log entries in an input context, and their pre-training corpora include cybersecurity documentation — the MITRE ATT&CK catalogue, threat-intelligence reports, vendor advisories — which equips the model with a broad base of security domain knowledge. Pre-training alone, however, is not sufficient, since the deployment’s specific log formats and target technique catalogue fall outside the pre-training distribution. Fine-tuning on log-to-technique supervision supplies the missing mapping, aligning the latent knowledge with the deployment specifics.

In the proposed framework, the fine-tuned LLM serves as the classifier of Stage 2 (Chap-

ter 4); the training procedure is detailed in Chapter 5.

2.5 Related Works

Prior work on automated attack detection from system evidence varies along three orthogonal axes that together determine how complete a detection pipeline is: the input modalities it consumes, the granularity of its output, and the methodology it adopts. The same three axes structure Table 1. The Input columns indicate whether a system consumes endpoint logs, network logs, alerts, or some combination of these; the Output columns indicate whether the system stops at alert filtering, tactic-level classification, technique-level classification, or full lifecycle reconstruction; and the Approach columns indicate whether the system relies on causal-graph traversal, hand-crafted rules, classical machine learning, deep learning, or large language models. The bottom row of the table shows that the framework proposed in this thesis is the only system that occupies all three axes simultaneously: multi-source input, lifecycle-level output, and a hybrid causal-graph + LLM approach. The remainder of this section walks through the prior works in increasing order of output granularity, contrasting each against these axes.

Table 1: Comparison of related works on attack detection and lifecycle reconstruction.

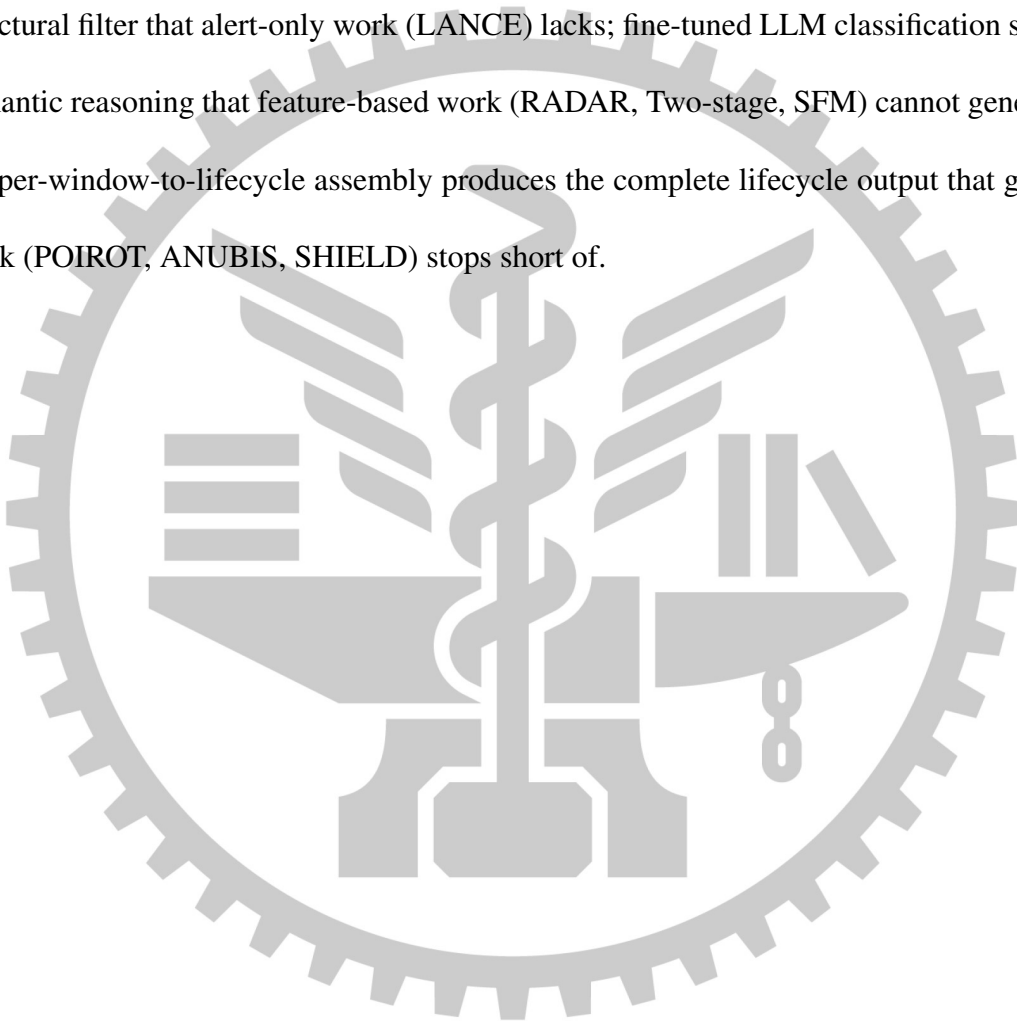
Paper	Year	Input			Output				Approach				
		System Log	Network Log	Alert	Alert	Tactic	Technique	Lifecycle	Causal Graph	Rule	ML	DL	LLM
POIROT [12]	2019	V			V				V	V			
ANUBIS [13]	2022	V			V				V			V	
Graph-Based APT [14]	2021	V		V	V				V	V		V	
SHIELD [17]	2025	V			V	V			V				V
RADAR [15]	2024		V				V			V	V		
Two-stage [16]	2024	V	V				V				V		
SFM [19]	2025	V					V					V	
LANCE [18]	2025			V			V						V
Ours	2026	V	V	V			V	V	V				V

The earliest line of work stops at coarse outputs. Milajerdi et al. [11] proposed POIROT, which formalises threat hunting as inexact graph pattern matching between manually constructed CTI query graphs and provenance graphs built from kernel audit logs. POIROT demonstrated effective detection on DARPA TC scenarios but operates at the campaign level: each match yields a single alert and leaves the constituent techniques implicit. Anjum et al. [12] developed ANUBIS, which combined LSTM and Bayesian Neural Networks on provenance graphs and achieved strong detection performance on DARPA OpTC; its output, however, is a binary anomaly flag rather than an ATT&CK technique. Fan et al. [13] applied GraphSAGE to provenance graphs and substantially reduced EDR false alarms, but ATT&CK technique identifiers were used as input features for graph learning rather than as detection outputs — the system presupposes that technique labels are already available. Gandhi et al. [14] proposed SHIELD, which combined Louvain community detection with LLM-driven chain-of-thought reasoning on audit logs; SHIELD reported high detection precision on DARPA CADETS but maps detected activity to tactic-level kill-chain stages rather than to individual techniques.

A second cluster of works reaches technique-level output but is bounded by one of the other two axes. Sharma et al. [15] developed RADAR, which uses rule-based features and per-technique decision trees on network flows to detect a fixed set of ATT&CK techniques; RADAR is limited to network traffic only, missing every host-level activity. Lin et al. [16] proposed a two-stage XGBoost approach on multi-source logs; although it achieves strong classification scores, its reliance on hand-crafted features over static log templates limits generalisation to previously unseen technique implementations. Huang et al. [17] proposed SFM, which applies SecureBERT embeddings and a BiGRU-CRF sequence labeller to real APT traces; SFM is designed to cope with severely imbalanced labels but requires labour-intensive event-level BIO annotation and exhibits lower precision in complex environments. Lin et al. [18] pro-

posed LANCE, applying a LoRA-fine-tuned LLM to multi-sensor alerts for lifecycle prediction; LANCE demonstrates LLM-based semantic reasoning for forensics, but its sole input is alert data — and because alerts miss a substantial fraction of the techniques actually executed, its coverage is fundamentally bounded by alert completeness.

None of these systems occupies all three axes at once. The framework proposed in this thesis integrates them: causal-graph traversal across endpoint and network logs supplies the structural filter that alert-only work (LANCE) lacks; fine-tuned LLM classification supplies the semantic reasoning that feature-based work (RADAR, Two-stage, SFM) cannot generalise; and the per-window-to-lifecycle assembly produces the complete lifecycle output that graph-based work (POIROT, ANUBIS, SHIELD) stops short of.



Chapter 3.

Problem Formulation

This chapter formalizes the two coupled sub-problems that together constitute the lifecycle completion task.

3.1 Notation

Table 2 summarizes the symbols used throughout this thesis. They fall into three categories: (1) Alerts and Logs, which define the input data sources; (2) Attack Lifecycle, which covers the partial lifecycle given as input, the completed lifecycle produced by the framework, and the ground-truth lifecycle used for evaluation; and (3) Graph, which describes the causal graph structure central to the extraction method.

3.2 Problem Statement

For each attack scenario, the framework receives two collections: a set of raw logs recording host and network activity, and a set of alerts produced by security monitoring tools applying predefined detection rules to that activity. The framework operates offline on a fixed input rather than a continuously growing stream. Its goal is to recover the ordered sequence of MITRE ATT&CK techniques employed during the attack. Because the detection rules cover only a fraction of the technique universe, the lifecycle derived from alerts alone is incomplete; the

Table 2: Summary of notations.

Category	Notation	Description
Alerts & Logs	$A = \langle a_1, a_2, \dots \rangle$	Alert sequence from EDR and NDR, sorted by timestamp, where a_i is the i -th alert
	$L = \langle l_1, l_2, \dots \rangle$	Raw log sequence from all sources, sorted by timestamp, where l_i is the i -th log
	$\hat{L} = \langle \hat{l}_1, \hat{l}_2, \dots \rangle$	Extracted attack-relevant log subsequence, $\hat{L} \subseteq L$, where $\hat{l}_i$ is the i -th extracted attack-relevant log
	$L^* = \langle l_1^*, l_2^*, \dots \rangle$	Ground-truth attack-relevant log sequence, where l_i^* is the i -th ground-truth attack-relevant log
Attack Lifecycle	t_i	The i -th technique in the lifecycle
	$LC_P = \langle t_1, t_2, \dots, t_i \rangle$	Partial lifecycle derived from alerts, treated as given input to the framework
	$LC_C = \langle \hat{t}_1, \hat{t}_2, \dots, \hat{t}_j \rangle$	Completed lifecycle
	$LC_{GT} = \langle t_1^*, t_2^*, \dots, t_k^* \rangle$	Ground-truth lifecycle
	$MITRE$	MITRE ATT&CK Technique universe
Graph	V	Node set — processes, files, IP sockets
	E	Directed edge set — causal events between nodes
	$G = (V, E)$	Causal graph constructed from logs
	$G' = (V', E')$	Attack subgraph, where $V' \subseteq V$ and $E' \subseteq E$

missing techniques leave traces only in the raw logs, which are orders of magnitude larger than the alert stream and carry no technique labels. Reconstructing the complete lifecycle therefore decomposes into two coupled sub-problems: first, isolating the small subset of raw logs that pertains to the attack from the much larger volume of benign activity; and second, combining the isolated evidence with the partial lifecycle LC_P obtained from those alerts into a completed sequence of techniques that approximates the ground-truth lifecycle.

The two sub-problems are now stated formally using the notation of Section 3.1. Three assumptions hold throughout: (1) L is captured completely and is not tampered; (2) every technique actually employed during the attack leaves at least one observable trace in $L \cup A$; and (3) the partial lifecycle LC_P is given to the framework as an input alongside L and A . The procedure that produces LC_P from the alert stream is a deterministic preprocessing step, described in Chapter 5, rather than a sub-problem this thesis sets out to solve.

Sub-problem 1 — Attack-Relevant Log Extraction. Given L and A , produce $\hat{L} \subseteq L$ that maximizes the F1-score against the ground-truth attack-relevant log set L^* . Letting precision and recall be defined over set membership,

$$P = \frac{|\hat{L} \cap L^*|}{|\hat{L}|}, \quad R = \frac{|\hat{L} \cap L^*|}{|L^*|}, \quad F_1 = \frac{2PR}{P + R}. \quad (3.1)$$

Sub-problem 2 — Lifecycle Completion. Given $\hat{L}$ and LC_P , produce LC_C that maximizes the order-aware Lifecycle LCS-F1 against LC_{GT} . Let $\ell = |\text{LCS}(LC_C, LC_{GT})|$ denote the length of the longest common subsequence between the completed and ground-truth lifecycles; the objective is

$$P = \frac{\ell}{|LC_C|}, \quad R = \frac{\ell}{|LC_{GT}|}, \quad \text{LCS-F1} = \frac{2PR}{P + R}, \quad (3.2)$$

subject to $LC_P \subseteq LC_C \subseteq \text{MITRE}$: every alert-detected technique is preserved, and every technique is drawn from the MITRE ATT&CK universe.

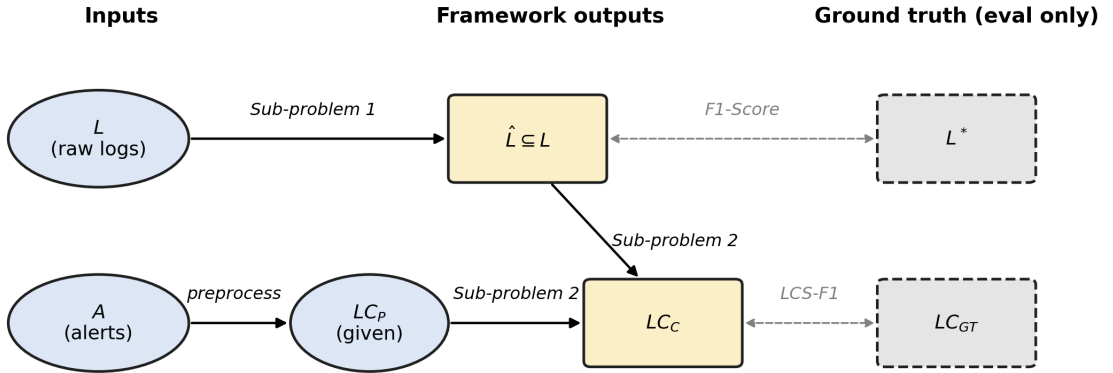
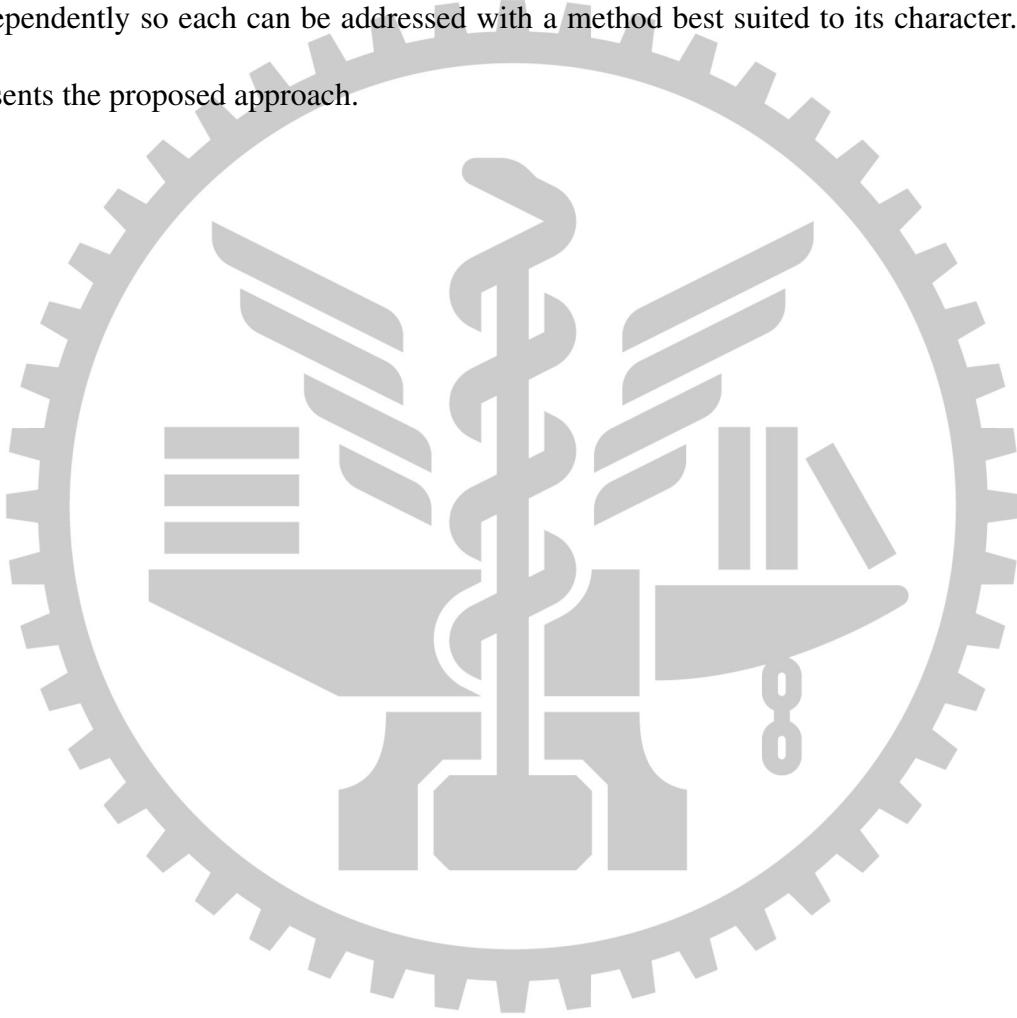


Figure 4: Schema of the problem formulation. Solid arrows denote operations performed by the framework; dashed double arrows denote evaluation that compares framework outputs against ground truth (used only offline).

Figure 4 visualizes the formulation. The inputs L , A , and LC_P appear on the left; the partial lifecycle LC_P is obtained from A by the deterministic preprocessing step of Assumption 3 and enters the framework as an input rather than as an output to be computed. The two framework outputs — the extracted log set $\hat{L}$ and the completed lifecycle LC_C — occupy the centre. The

ground-truth sets L^* and LC_{GT} , drawn dashed on the right, are used only for offline evaluation and are not visible to the framework at inference time. Sub-problem 1 maps L to $\hat{L}$ and is evaluated by F1 against L^* . Sub-problem 2 combines LC_P with $\hat{L}$, used as semantic evidence, to produce LC_C , which is evaluated by LCS-F1 against LC_{GT} . The diagonal arrow from $\hat{L}$ to LC_C captures the coupling that motivates the two-stage design: the quality of the completed lifecycle is bounded by that of the extracted evidence, yet the two sub-problems are formulated independently so each can be addressed with a method best suited to its character. Chapter 4 presents the proposed approach.



Chapter 4.

Solution Approach

This chapter presents the proposed two-stage framework for completing attack lifecycle. Section 4.1 provides the solution overview. Section 4.2 details Stage 1 — Graph-based Log Extraction, which addresses Sub-problem 1 of Chapter 3. Section 4.3 describes Stage 2 — LLM-Driven Lifecycle Completion, which addresses Sub-problem 2. Section 4.4 walks through a representative example.

4.1 Solution Overview

The proposed framework accepts three categories of input: raw logs L from endpoint and network loggers, security alerts A from EDR and NDR (each alert carries a severity score), and a partial lifecycle LC_P derived from those alerts. The framework produces a single output: the completed lifecycle LC_C that fills the gaps in LC_P . Figure 5 shows the overall architecture.

In Stage 1, *Log2Graph* constructs a causal graph $G = (V, E)$ from the heterogeneous raw logs. *Attack Subgraph Search* then identifies the attack-relevant subgraph $G' = (V', E')$ using alert-driven malicious scoring and breadth-first search, and *Graph2Log* maps G' back to original log entries, producing the filtered attack-relevant log set $\hat{L}$. In Stage 2, the alert-detected techniques in LC_P split the timeline into alert-gaps; each alert-gap is further partitioned into fixed-length time windows, and a fine-tuned multi-label LLM classifies each window. The per-

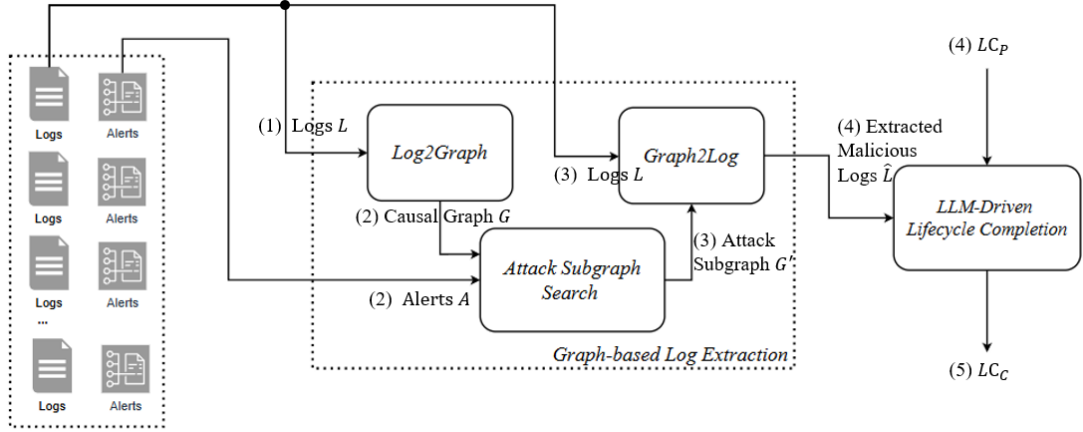


Figure 5: Solution overview.

window predictions are concatenated, consecutively deduplicated, and interleaved with LC_P to form LC_C .

The two-stage design reflects a separation of concerns. Stage 1 addresses the log imbalance problem through structural graph-based filtering. Stage 2 addresses the semantic gap problem through LLM-driven technique identification.

4.2 Graph-based Log Extraction

Raw logs are a flat stream of discrete records, but the events they describe are not independent. Each entry records that one system entity acted on another — a process spawned another, a process wrote a file, a process opened a network connection — and thereby encodes a cause-effect relation between two entities. Log entries that share an entity reference the same causal context. A causal graph makes this structure explicit: nodes are system entities, edges are the cause-effect relations carried by individual log entries. An attack is a chain of causally connected actions across the entities the attacker controls, so it manifests in the graph as a connected subgraph. Given any alert that identifies a node within this subgraph, graph traversal recovers the full chain, and the corresponding log entries form the attack-relevant subset $\hat{L}$.

Stage 1 realizes this strategy as three modules. *Log2Graph* (Section 4.2.1) converts raw logs

L into the causal graph $G = (V, E)$. *Attack Subgraph Search* (Section 4.2.2) extracts the attack subgraph $G' = (V', E')$ from G using alerts as guidance. *Graph2Log* (Section 4.2.3) maps G' back to the original log entries to produce $\hat{L}$.

4.2.1 Log2Graph

The construction proceeds in two phases: a backbone is first built from endpoint events, and a network-event overlay is then attached to it. The graph uses three node types — process, file, and socket — identified respectively by ProcessGuid, full file path, and IP:Port. Directed edges encode the cause-effect relations carried by individual log entries, and each recognised event type maps to a specific edge type connecting a specific pair of node types. Table 3 lists the full schema.

In the first phase, most edges are read directly from endpoint events: each event produces exactly one edge between the two entities it references, of a type determined by the event type. The exception is the `File_to_Proc_Exec` edge, which is derived: whenever a process’s image path matches an existing file node, an edge is added from that file node to the process, linking file delivery to subsequent process execution. This derived edge plays a critical role in attack entry tracing (Section 4.2.2).

The second phase overlays network events onto the backbone. When a network event (HTTP, TLS, flow, or SSH) references an IP:Port already present from an endpoint event, it is attached to the corresponding socket node, bridging endpoint-level and network-level views. This bridge is essential: endpoint logs record which process initiated a connection but not the application-layer protocol, while network logs capture protocol details but not the originating process. Throughout both phases, every edge retains the identifier of the log entry that produced it; since each log entry maps to exactly one edge, this one-to-one correspondence enables

Graph2Log (Section 4.2.3) to losslessly recover the original log entries from any subgraph.

Table 3: Causal graph node types and edge types.

Category	Name	Type	Description
Node	Process	Process	Execution context, identified by ProcessGuid (with image, command line, user)
Node	File	File	File system object, identified by full file path
Node	Socket	Socket	Network endpoint, identified by IP:Port
Edge	E1	Proc→Proc	Parent creates child process (endpoint EID 1)
Edge	E2	Proc→File	File creation-time modification (endpoint EID 2)
Edge	E3	Proc→Socket	Network connection (endpoint EID 3)
Edge	E6	Proc→File	Driver loaded (endpoint EID 6)
Edge	E7	Proc→File	Image / DLL loaded (endpoint EID 7)
Edge	E8	Proc→Proc	Remote thread injection (endpoint EID 8)
Edge	E10	Proc→Proc	Process access (endpoint EID 10)
Edge	E11	Proc→File	File creation / write (endpoint EID 11)
Edge	E23	Proc→File	File delete (endpoint EID 23)
Edge	Exec	File→Proc	File executed as process (derived from endpoint EID 1)
Edge	Link	Socket→Socket	HTTP / TLS / flow / SSH (network logger)

4.2.2 Attack Subgraph Search

This module extracts the attack subgraph $G' = (V', E')$ from G using alerts A as guidance. The strategy proceeds from the inside out: first, identify the most suspicious processes from alert evidence; then trace backward to find where the attack entered the system; finally, expand forward from each entry point and prune branches that lack alert evidence, capturing the full attack scope while excluding benign activity. The three steps are formalized in Algorithms 1–3.

Step 1 — Malicious Seed Node Identification. Each alert references one or more process nodes in G ; let $refs(a) \subseteq V$ denote the set of process nodes referenced by alert a . The algorithm scores each process v by accumulating the severity of every alert that references it:

$$MaliciousScore(v) = \sum_{\substack{a \in A \\ v \in refs(a)}} severity(a). \quad (4.1)$$

The top- K scored processes are selected as seed nodes, focusing the subsequent search on the processes with the strongest cumulative alert evidence and avoiding propagation from processes with only weak or incidental alert associations.

Algorithm 1 Malicious Seed Identification

Input: Causal graph $G = (V, E)$, alert set A , parameter K

Output: Alerted process set R , seed set S

```

1: for each alert  $a \in A$  do
2:   for each process node  $v$  referenced by  $a$  do
3:      $score[v] \leftarrow score[v] + severity(a)$ 
4:   end for
5: end for
6:  $R \leftarrow \{v \in V \mid score[v] > 0\}$  # all alerted processes
7:  $S \leftarrow$  top- $K$  nodes from  $R$  ranked by  $score$ 
8: return  $R, S$ 

```

Step 2 — Entry Node Tracing. From each seed node, the algorithm traces backward through the causal graph to identify the attack entry node — the earliest attacker-controlled process in the attack chain. The traversal follows two paths from the current process. The direct path moves one hop back along a process-to-process edge to the parent process. The file-mediated path moves two hops: along a file-to-process edge back to the source file from which the current process was loaded, then along a process-to-file edge back to the process that wrote that file. The file-mediated path is critical when an attacker drops a payload that an OS service later executes: the direct lineage is severed at the service boundary, but the file-mediated path reconnects the execution chain to the original delivery chain. Subsystem processes (`services.exe`, `svchost.exe`, etc.) are filtered out only on the direct path, since they mark a trust boundary the attacker does not own; on the file-mediated path they are kept, because a file that any process loaded — subsystem or not — must still have been written by some earlier process, and that writer is the relevant predecessor. The traversal stops when the current process has no causal predecessor of either kind, and that process is recorded as an entry node. Termination is guaranteed because G is acyclic by construction: edges record causal events in

timestamp order, so following any predecessor strictly decreases time.

Algorithm 2 Entry Node Tracing

Input: Causal graph G , seed set S , OS subsystem process set $Subsystem$

Output: Entry node set $entries$

```

1:  $entries \leftarrow \emptyset$ ;  $queue \leftarrow S$ ;  $visited \leftarrow \emptyset$ 
2: while  $queue$  is not empty do
3:   take a node  $n$  from  $queue$ 
4:   if  $n \in visited$  then
5:     continue
6:   end if
7:    $visited \leftarrow visited \cup \{n\}$ 
8:    $direct \leftarrow \text{Proc} \rightarrow \text{Proc}$  parents of  $n$  not in  $Subsystem$       # direct path
9:    $bridge \leftarrow$  processes  $p$  with  $p \rightarrow f \rightarrow n$  for some file  $f$   # file-mediated path
10:   $predecessors \leftarrow direct \cup bridge$ 
11:  if  $predecessors = \emptyset$  then
12:     $entries \leftarrow entries \cup \{n\}$                                 # attack entry found
13:  else
14:     $queue \leftarrow queue \cup predecessors$ 
15:  end if
16: end while
17: return  $entries$ 

```

Step 3 — Forward Expansion with Branch Pruning. Step 2 locates where the attack entered the system; Step 3 determines the full scope of its downstream impact. The challenge is that entry nodes are typically long-running processes that spawn both malicious and routine children. The algorithm first performs a breadth-first search from each entry node, collecting all reachable descendants and their associated non-process nodes. It then prunes each direct child branch independently: if no process in a branch was ever referenced by any alert, the entire branch is classified as benign and removed. An alert against any process in a branch is a high-precision indicator that the branch is attack-related, while a branch with no alerted process at all is very likely benign; branch-level rather than per-process granularity preserves intermediate processes that may lie between alerted ones in an attack chain. The pruning criterion considers all processes with $MaliciousScore > 0$, not just the top- K seeds from Step 1, so that even a single low-severity alert within a branch is sufficient to preserve it. Pruning operates on

the process-creation subtree of each entry; injection and access edges, which cross subtree boundaries, are not used to delimit branches because they would conflate independent attack lineages. The remaining nodes and edges form $G' = (V', E')$.

Algorithm 3 Forward Expansion with Branch Pruning

Input: Causal graph $G = (V, E)$, entry node set *entries*, alerted-process set R

Output: Attack subgraph $G' = (V', E')$

```

1:  $V' \leftarrow \emptyset$ 
2: for each entry node  $e \in \text{entries}$  do
3:   add to  $V'$  every node reachable from  $e$  in  $G$            # (a) downstream of  $e$ 
4:   for each child  $c$  with edge  $e \rightarrow c$  of type  $\text{Proc} \rightarrow \text{Proc}$  do
5:      $T \leftarrow$  process subtree rooted at  $c$            # via  $\text{Proc} \rightarrow \text{Proc}$  only
6:     if  $T \cap R = \emptyset$  then                         # no alert in branch
7:       remove every node of  $T$  from  $V'$                  # (b) drop benign branch
8:     end if
9:   end for
10: end for
11:  $E' \leftarrow$  all edges  $(u, v) \in E$  with  $u \in V'$  and  $v \in V'$ 
12: return  $G' = (V', E')$ 

```

4.2.3 Graph2Log

Graph2Log is the inverse of *Log2Graph*: it converts the attack subgraph G' back to original log entries, producing the extracted attack-relevant log set $\hat{L}$.

Returning to log text aligns Stage 1’s output with how Stage 2 consumes input. A large language model is a sequence model: its native input modality is text, and its pre-training corpora contain extensive system-log content, so the format-specific cues that distinguish technique signatures — command-line arguments, file paths, protocol-layer fields — fall within its pre-training distribution. A graph representation would have to be serialized into adjacency lists or edge-by-edge enumerations, a format the model has not been pre-trained on; serialization also discards the fine-grained log fields on which technique identification depends. Stage 2 therefore operates on log text rather than on G' directly.

As described in Section 4.2.1, each edge in G retains the identifier of the log entry that

produced it. *Graph2Log* collects these identifiers from all edges in G' , retrieves the corresponding entries from L , and sorts them by timestamp to produce $\hat{L}$. Because the correspondence between log entries and edges is one-to-one, the back-mapping is lossless: every log entry that contributed to G' is recovered in full.

4.3 LLM-Driven Lifecycle Completion

This section details Stage 2, which addresses Sub-problem 2: given the extracted attack-relevant logs $\hat{L}$ and the partial lifecycle LC_P , recover the techniques missing from LC_P and assemble them with LC_P to produce the completed lifecycle LC_C .

Mapping log entries to ATT&CK techniques is a semantic problem rather than a syntactic one: a single log line can support different technique labels depending on context, and a single technique can be realized by very different log lines. Resolving this ambiguity requires prior knowledge about what each technique means, and that knowledge resides in the natural-language documentation accompanying the ATT&CK framework rather than in any individual log entry. A large language model is pre-trained on a corpus that includes this documentation, so the technique-level semantics already lives in its parameters. Fine-tuning on log-to-technique supervision then aligns this latent knowledge with the deployment's log formats and technique catalogue. The fine-tuned LLM is the foundation on which the rest of Stage 2 assembles the completed lifecycle LC_C .

The remaining design challenge is that the alert-gap between two consecutive alert-detected techniques in LC_P may itself contain log events from several missing techniques. Passing all events of such an alert-gap to the LLM in one invocation would force the model to identify multiple techniques from a long log context, increasing the risk of conflation and hallucination. The proposed approach therefore partitions the timeline twice. The N alert-detected techniques in

LC_P first split the timeline into $N + 1$ alert-gaps — one before the earliest detected technique, one between each consecutive pair, and one after the latest — each alert-gap being a contiguous span where missing techniques may reside. Within an alert-gap, the logs from $\hat{L}$ are then partitioned into fixed-length time windows of W seconds. The default $W = 5$ s is small enough that most windows contain evidence from at most one technique, while occasional multi-technique windows are still handled because the classifier is trained as a multi-label one.

Each window is passed independently to a fine-tuned LLM whose output is a JSON object listing the ATT&CK techniques observed in that window. The classifier operates over a fixed vocabulary of 34 ATT&CK techniques present in the dataset together with a sentinel BENIGN class for windows in which no attack technique is active, giving 35 classes in total. Treating each window as a multi-label problem rather than forcing a single-technique choice avoids the failure mode in which two genuinely co-occurring techniques are collapsed into one, and lets the model emit BENIGN for windows that contain only routine activity that happened to survive Stage 1. The training data, prompt format, and decoding configuration that realize this classifier are detailed in Chapter 5.

Within each alert-gap, the per-window predictions are aggregated in two passes. The first pass concatenates the window outputs in chronological order, producing a flat sequence of technique identifiers. The second pass applies a consecutive-duplicate suppression: whenever the same identifier appears in adjacent positions of the flat sequence, only the first occurrence is retained. This collapses the long runs of repeated identifiers that arise when a single technique executes across several consecutive windows, while preserving repeated executions of the same technique that are separated by other techniques. BENIGN markers are then dropped, because the goal at the lifecycle level is to enumerate the missing attack techniques, not to record where no attack was observed.

The alert-gap predictions are finally interleaved with the alert-detected techniques of LC_P to form LC_C : the prediction of alert-gap 1 is appended, then t_1 (the first technique of LC_P), then the prediction of alert-gap 2, and so on. Because no element of LC_P is ever removed, the constraint $LC_P \subseteq LC_C$ from Sub-problem 2 holds by construction; new techniques can only be inserted between consecutive elements of LC_P . Algorithm 4 summarizes the full procedure.

Algorithm 4 Stage 2 Lifecycle Completion

Input: Attack-relevant logs $\hat{L}$, partial lifecycle $LC_P = \langle t_1, \dots, t_N \rangle$, window size W

Output: Completed lifecycle LC_C

```

1:  $gaps \leftarrow$  partition  $\hat{L}$  at the time-ranges of  $LC_P$            #  $N+1$  alert-gaps
2:  $preds \leftarrow []$                                            # one list per alert-gap
3: for  $k = 1$  to  $N + 1$  do
4:    $windows \leftarrow$  split  $gaps[k]$  into consecutive  $W$ -second windows
5:    $seq \leftarrow []$ 
6:   for each window  $w \in windows$  in chronological order do
7:      $L_w \leftarrow \text{LLM\_CLASSIFY}(w)$            # [BENIGN] if no technique
8:     append every element of  $L_w$  to  $seq$ 
9:   end for
10:  collapse every run of consecutive duplicates in  $seq$ 
11:  remove every BENIGN entry from  $seq$ 
12:   $preds[k] \leftarrow seq$ 
13: end for
14:  $LC_C \leftarrow []$                                            # interleave  $preds$  with  $LC_P$ 
15: for  $k = 1$  to  $N + 1$  do
16:   append every element of  $preds[k]$  to  $LC_C$ 
17:   if  $k \leq N$  then
18:     append  $t_k$  to  $LC_C$            #  $k$ -th technique of  $LC_P$ 
19:   end if
20: end for
21: return  $LC_C$ 

```

4.4 Case Study

This section traces the framework end-to-end on the scenario introduced in Section 2.1 (Figure 1). When the attack is replayed on the testbed, it generates 57,832 raw logs. The replay also generates 40 EDR/NDR alerts, of which 38 carry MITRE technique tags. Sorting the alerts

by timestamp and collapsing consecutive duplicate tags yields

$$LC_P = \langle T1059.001, T1059.003 \rangle,$$

covering two of the seven distinct techniques in the ground-truth lifecycle. The framework must recover the other five — T1071.001, T1566.002, T1620, T1134, and T1490 — from the surrounding noise.

Stage 1 converts the 57,832 raw entries into a causal graph and then prunes it to the attack subgraph. Alert severity is summed onto each referenced process node (Algorithm 1); the alert-flagged PowerShell process emerges as the top-scoring seed. Backward entry tracing reaches the attacker’s inbound SSH session — the channel through which the attack was orchestrated — and marks it as the attack entry. Forward expansion then fans out from the entry while pruning branches that contain no alerted process. *Graph2Log* reads the log identifier on each retained edge to recover the attack-relevant log set $\hat{L}$, reducing the input from 57,832 entries down to roughly 90 while preserving the clusters that the ground truth identifies as malicious.

Stage 2 partitions the timeline at the two alert-detected techniques in LC_P , producing three alert-gaps. Figure 6 overlays this partitioning on the lifecycle of Figure 1: the two green boxes are the alert-detected techniques at 22:19:43 (T1059.001) and 22:21:32 (T1059.003), and the dashed yellow regions are the resulting Alert-Gaps 1, 2, and 3 along the timeline shown beneath the boxes. Each gap is sliced into 5-second windows that the fine-tuned multi-label LLM classifies independently.

Alert-Gap 1 yields $\langle T1071.001, T1566.002 \rangle$ from windows containing the inbound C2 channel and the phishing-link payload download. Alert-Gap 2 yields $\langle T1620, T1071.001, T1134 \rangle$ from windows containing the reflective DLL load that follows the PowerShell invocation, a return C2 beacon, and a remote-thread injection into a privileged Windows service. Alert-Gap 3,

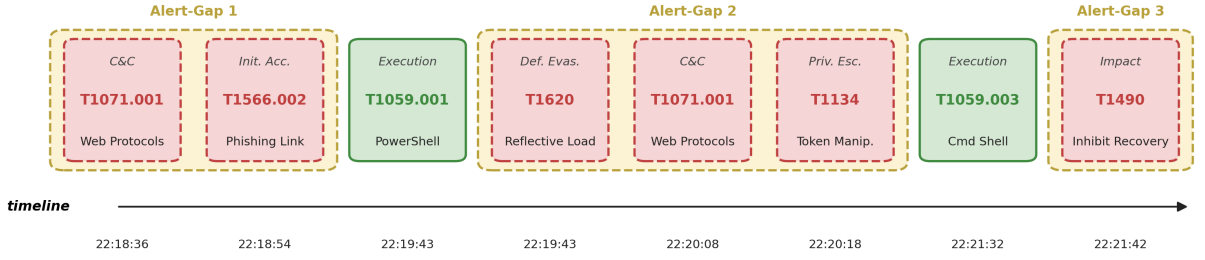


Figure 6: Alert-gap partitioning of the example scenario. The two alert-detected techniques in LC_P (green boxes) split the eight-event lifecycle into three alert-gaps (dashed yellow regions) along the timeline shown beneath the boxes. Each alert-gap is sliced into 5-second windows for Stage 2 inference.

which holds a single ground-truth event, illustrates the per-window inference most cleanly: the window covering 22:21:42–22:21:47 contains a process-creation event in which `cmd.exe` launches `schtasks.exe` with `/Change /disable` against the Windows System Restore scheduled task, and the model returns [T1490], citing the `schtasks /Change` command and the System-Restore target as evidence; the other windows in this gap hold only routine activity that survived Stage 1’s pruning, and the model marks them BENIGN.

Interleaving these three alert-gap predictions with the two alert-detected techniques in LC_P produces

$$LC_C = \langle T1071.001, T1566.002, T1059.001, T1620, \\ T1071.001, T1134, T1059.003, T1490 \rangle,$$

which contains all eight ground-truth events in correct temporal order. The six techniques previously missing from LC_P have been recovered from log evidence alone, closing the alert gap shown in Section 2.1.

Chapter 5.

Implementation

This chapter describes the implementation of the proposed framework. Section 5.1 presents the experimental environment, including the testbed architecture and the tools used. Section 5.2 describes the data collection process, covering the attack scenarios, log collection, and partial lifecycle derivation. Section 5.3 details the implementation of Stage 1 (Graph-based Log Extraction). Section 5.4 details the implementation of Stage 2 (LLM-Driven Lifecycle Completion), including model selection, fine-tuning, and the inference pipeline.

5.1 Experimental Environment

The testbed consists of three virtual machines connected within the same network: (1) a Kali Linux machine serving as the attacker, running the Sliver C2 framework to orchestrate attack scenarios; (2) a Windows Server 2008 R2 machine serving as the victim, instrumented with Sysmon for endpoint logging and Suricata for network traffic logging; and (3) a Wazuh server that collects logs from the victim and generates security alerts based on predefined detection rules. Sysmon captures process creation, file operations, network connections, and other system events as structured XML logs. Suricata operates as a network intrusion detection system, producing eve.json logs that record HTTP, TLS, DNS, SSH, and flow-level events. Wazuh correlates both Sysmon and Suricata events against its rule set and produces alerts with severity

levels mapped to MITRE ATT&CK techniques.

Table 4 summarizes the open-source tools and libraries used throughout the implementation.

Table 4: Open-source tools and libraries.

Category	Name	Purpose
Graph Processing	NetworkX [19]	Directed graph construction and traversal
Deep Learning	PyTorch	Model training and inference backend
	HuggingFace [20]	Model loading and tokenization
	PEFT	LoRA [21] adapter for parameter-efficient fine-tuning
	Unsloth ¹	Accelerated fine-tuning and inference
Log/Alert Collection	Sysmon	Windows system event logging
	Suricata	Network intrusion detection
	Wazuh	Security alert generation and log correlation
Attack Emulation	Sliver	C2 framework for attack orchestration
	Aurora	Attack scenario dataset and replay scripts [22]

5.2 Data Collection

5.2.1 Attack Scenarios and Log Collection

The dataset is generated using Aurora [22], a cyberattack emulation framework that provides 250 attack scenarios derived from real-world Cyber Threat Intelligence (CTI) reports. Of these, 13 scenarios rely on Metasploit modules that are incompatible with the Windows Server 2008 R2 victim and are excluded, leaving 237 scenarios used throughout this work. Each scenario is defined as a YAML specification containing an ordered sequence of attack actions, where each action is annotated with its MITRE ATT&CK technique ID, the executor type (e.g., PowerShell, command shell, C2 built-in), and the command to be executed. The 237 retained scenarios collectively cover 34 unique ATT&CK techniques spanning 10 tactics, including Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Dis-

¹<https://github.com/unslothai/unsloth>

covery, Collection, Lateral Movement, Command and Control, and Impact.

Each scenario is replayed on the testbed using Aurora’s semi-automatic replay script, which reads the YAML specification and orchestrates the attack through the Sliver C2 framework. The script automates most attack steps but requires human intervention for certain actions that cannot be fully scripted (e.g., initial phishing interactions). During execution, the script records the start and end timestamps of each step for ground-truth labeling, while Sysmon and Suricata continuously collect endpoint and network logs on the victim machine and Wazuh generates alerts based on observed events. After each scenario completes, the collected logs are labeled by matching their timestamps against the recorded step boundaries: a log entry falling within the time window of an attack step is labeled as malicious and annotated with the corresponding technique ID; all other entries are labeled as benign. The labeled logs constitute the ground-truth attack-relevant log set L^* . The ground-truth lifecycle LC_{GT} is derived directly from the YAML specification, which defines the ordered sequence of techniques for each scenario.

5.2.2 Partial Lifecycle Derivation

The partial lifecycle LC_P is derived from the alert set A produced by Wazuh and Suricata. Each alert that maps to a MITRE ATT&CK technique contributes its technique ID and timestamp to LC_P . Alerts are sorted by timestamp, and for each alert that carries a technique mapping, the technique ID is appended to LC_P if it is not already present as the most recent entry.

The resulting LC_P is an ordered sequence of distinct techniques reflecting only what the detection rules were able to identify; each entry retains the time range covered by its originating alerts, which Stage 2 later uses to define alert-gap boundaries. Across the 237 scenarios, LC_P covers approximately 40% of the ground-truth techniques on average, confirming the alert

coverage gap that motivates this thesis.

5.3 Stage 1: Graph-based Log Extraction

Stage 1 is implemented in Python using NetworkX for graph construction and traversal. The implementation follows the three-module pipeline described in Chapter 4.

Log2Graph first parses Sysmon XML logs into structured JSON records, extracting fields relevant to each EventId (e.g., ProcessGuid, ParentProcessGuid, Image, CommandLine for process events; SourceIp, DestinationIp for network events). Suricata eve.json logs are similarly normalized. The parsed events are then loaded into a NetworkX directed graph (DiGraph), where each event creates one or more nodes and a directed edge with the event's metadata and log identifier stored as edge attributes. When multiple events produce the same source-destination pair, the implementation appends to a list of log identifiers on the existing edge rather than overwriting, preserving the one-to-one back-mapping capability.

Attack Subgraph Search loads the Wazuh alert JSON and computes the *MaliciousScore* for each process node by summing the `rule.level` field across all alerts referencing that node's ProcessGuid. The top- K scored processes are selected as seed nodes; the value of K used in experiments is given in Section 6.1. Entry node tracing terminates at a predefined set of Windows subsystem processes (services.exe, svchost.exe, lsass.exe, csrss.exe, wininit.exe, smss.exe). Forward expansion uses a single breadth-first search per entry node, and branch pruning removes E1-subtrees containing no scored process.

Graph2Log iterates over all edges in the extracted subgraph, collects the stored log identifiers, retrieves the corresponding entries from the original parsed log files, and sorts the result by timestamp to produce $\hat{L}$.

5.4 Stage 2: LLM-Driven Lifecycle Completion

5.4.1 Model Selection and Fine-Tuning

The LLM used in Stage 2 is Qwen3.5-122B-A10B, a 122-billion-parameter mixture-of-experts (MoE) causal language model with 10 billion active parameters per forward pass. The model is loaded in 4-bit quantized form to fit within GPU memory constraints. Fine-tuning is performed using LoRA (Low-Rank Adaptation) applied to all attention and feed-forward projection layers (q_proj, k_proj, v_proj, o_proj, up_proj, down_proj, gate_proj); the rank and scaling factor used in experiments are given in Section 6.1. The Unsloth framework is used to accelerate both fine-tuning and inference.

Training data is constructed from the labeled dataset described in Section 5.2. For each attack scenario, the logs within each ground-truth technique’s time window are extracted, formatted with security-relevant fields (process name, command line, network endpoints), and paired with the expected output: a JSON object listing the technique IDs together with a rationale explaining the reasoning behind the classification. Benign training samples are constructed analogously from log windows that fall outside any ground-truth technique’s time range and are labelled [BENIGN]; they are down-sampled to maintain an approximately 4:1 ratio of malicious to benign samples. The fine-tuning is performed as reasoning-augmented SFT: by including the rationale in the training target, the model learns not only to predict the correct technique but also to produce an explicit chain of thought that connects observed log evidence to the technique definition. This approach improves classification accuracy by encouraging the model to ground its predictions in specific log artifacts rather than relying on surface-level pattern matching.

5.4.2 Inference Pipeline

At inference time, the partitioning and aggregation procedure of Section 4.3 (Algorithm 4) is applied to $\hat{L}$. The remainder of this subsection describes the implementation-level details: prompt construction, decoding, and output parsing. Hyperparameter values are given in Section 6.1.

For each window, log entries are formatted by selecting security-relevant fields per event type: process creation events include the process name, command line, parent process, and user; network events include source and destination addresses; file events include the target file path. The formatted entries are assembled into a single prompt and passed to the fine-tuned model. Inference uses nucleus sampling. The JSON output is parsed to obtain the window’s predicted list of technique IDs (or [BENIGN] when no attack technique is identified). These per-window predictions feed back into the aggregation procedure above to yield LC_C .

Chapter 6.

Evaluation and Results

This chapter evaluates the proposed framework presented in Chapter 4 and implemented as described in Chapter 5. Section 6.1 introduces the experimental setup, including dataset splits, baseline model configurations, evaluation metrics, and default hyperparameters. Section 6.2 profiles the dataset characteristics that shape the analyses that follow. Section 6.3 then presents the end-to-end headline result: how the framework compares against four baseline models on the lifecycle completion task. Section 6.4 isolates the contribution of three design factors. Section 6.5 finally characterizes the framework’s behavior under varying data conditions by stratifying performance along three complexity dimensions: sample, alert-gap, and lifecycle.

6.1 Experimental Setup

The evaluation uses the 237 scenarios retained from the dataset described in Section 5.2; they are partitioned at the scenario level into 190 train, 24 validation, and 23 test scenarios. Within each partition, fine-tuning samples are generated by sliding a window of $w \in \{1, 2, 3\}$ consecutive steps over each scenario’s ground-truth technique sequence; a window of width w yields one sample labeled with w techniques. The resulting dataset contains 15,198 / 1,900 / 1,580 train / validation / test samples (18,678 total). For lifecycle-level evaluation, the same 23 test scenarios are sliced into 173 alert-gaps by the alert-detected techniques in LC_P ; each

alert-gap is further subdivided into W -second windows for inference. Per-window detection and lifecycle completion are therefore two evaluation views over the same held-out scenarios, and train and validation scenarios contribute to neither.

Against these splits, the proposed framework is benchmarked against four baselines drawn from the model families likely to be considered for log-based lifecycle completion. Decision Tree (DT) serves as the classical machine-learning floor commonly adopted in prior log-based attack detection work. Bi-LSTM represents a sequence-aware deep-learning baseline used in similar settings. GPT-5 is queried without fine-tuning and provides a closed-source reference for what a substantially larger pre-trained model achieves out of the box. Qwen3.5-122B-A10B (Non-Finetuned) shares the backbone of the proposed model but omits the LoRA adapter, providing the controlled ablation that isolates the contribution of fine-tuning. The proposed model is Qwen3.5-122B-A10B (Finetuned), the LoRA-finetuned variant detailed in Section 5.4.1. All five models receive identical Stage 1 extracted logs $\hat{L}$ and identical alert-derived partial lifecycles LC_P unless otherwise stated. Table 5 lists the configuration of each model.

Table 5: Baseline and proposed-model configurations used throughout the evaluation.

Model	Type	Configuration
Decision Tree	Classical ML	TF-IDF features (max_features = 5000, n-gram range 1–2); CART criterion = gini, max_depth = none, min_samples_split = 2; scikit-learn 1.4
Bi-LSTM	Deep learning	Token embedding 128, hidden 256, 2 layers, dropout 0.3; sigmoid multi-label head; Adam, lr 10^{-3} , batch 32, 20 epochs, early stopping on validation Micro-F1
GPT-5 (Non-Finetuned)	Closed-source LLM	Public API; temperature 0.3, top- p 0.9, max output 1,536 tokens; identical prompt format to the Finetuned model
Qwen3.5-122B-A10B (Non-Finetuned)	Open-source LLM	4-bit quantized backbone; same prompt format and decoding parameters as the Finetuned variant; no LoRA adapter
Qwen3.5-122B-A10B (Finetuned)	Open-source LLM + LoRA	Same backbone; LoRA rank $r = 32$, scaling $\alpha = 64$ on all attention and FFN projections; reasoning-augmented SFT per Section 5.4.1

Performance is summarized by three metric families, two of which are standard and need no further definition: Micro-F1 and Macro-F1 for per-window technique detection (Section 6.5.1), and Precision, Recall, F1, and false-positive rate (FPR) for Stage 1 log extraction. The third family is LCS-F1, defined in Section 3.2 and order-aware because the order in which techniques are executed is part of the attack narrative. For convenience the definition is repeated: let $\ell = |\text{LCS}(LC_C, LC_{GT})|$ denote the length of the longest common subsequence between the predicted and ground-truth lifecycles; precision, recall, and LCS-F1 are

$$P = \frac{\ell}{|LC_C|}, \quad R = \frac{\ell}{|LC_{GT}|}, \quad \text{LCS-F1} = \frac{2PR}{P + R}. \quad (6.1)$$

LCS-F1 is reported under two views. *Per alert-gap LCS-F1* applies the formula above to the predicted and ground-truth technique sequences restricted to a single alert-gap and is used in the alert-gap-level analyses; *Lifecycle LCS-F1* applies it to the full LC_C and LC_{GT} , reflecting the framework’s overall ability to assemble the global attack narrative.

A small set of hyperparameters governs the framework’s behavior across all of the analyses that follow. Unless otherwise stated, the time-window size is $W = 5$ seconds and the seed count is $K = 3$; the LoRA, decoding, and output-length defaults appear in Table 5. Subsections that sweep a parameter state the swept range explicitly — Section 6.4.3, for example, sweeps W — while all other experiments inherit these defaults.

6.2 Data Profiling

This section characterizes the dataset along four dimensions that most directly shape the evaluation outcomes: ATT&CK coverage, lifecycle length, sample composition, and the share of the ground truth that alerts already supply. These dimensions provide the context in which

the headline, factor, and behavioral results are to be interpreted.

The 237 evaluation lifecycles span 34 unique ATT&CK techniques across the 10 tactics enumerated in Section 5.2.1. The technique-level distribution is long-tailed: a small subset of techniques accounts for a disproportionate share of step occurrences, while several techniques appear in only a handful of scenarios. Both classifiers and the LLM are therefore evaluated against an imbalanced label space, and the Macro-F1 reported alongside Micro-F1 in Section 6.5.1 is the metric designed to surface degradation on the rare end of this tail.

The lifecycles themselves are bounded in length: Aurora scenarios contain between 8 and 37 ATT&CK techniques per lifecycle, with median 25 and the bulk falling between 23 and 27. This concentration bounds the range over which the length-stratified analysis in Section 6.5.3 can claim length-invariance; any extrapolation to substantially longer attacks — for example, 50 techniques or more — lies outside what the present dataset can support.

At the sample level, the per-window samples mix three complexity strata by construction: $w = 1$ samples contain exactly one technique, $w = 2$ samples contain two co-occurring techniques, and $w = 3$ samples contain three. Single-technique samples dominate numerically because every scenario contributes more $w = 1$ windows than $w = 2$ or $w = 3$ windows, but the multi-technique samples are deliberately retained to expose the classifier to overlapping evidence. The behavioral analysis in Section 6.5.1 stratifies detection metrics along this composition.

Alert coverage completes the profile. As reported in Section 5.2.2, LC_P captures approximately 40% of the ground-truth techniques on average, leaving Stage 2 to recover the remaining 60%. Among the 173 test-set alert-gaps, 78 (45%) are benign with no ground-truth technique present, 30 contain one ground-truth technique, 26 contain two, 8 contain three, and 31 contain four or more. The alert-gap-level distribution is the substrate for the alert-gap-complexity

analysis in Section 6.5.2.

6.3 Lifecycle Completion: Finetuned LLM vs. Baseline Models

Table 6 reports Lifecycle LCS-F1 at the default window size $W = 5$ seconds for all five models. The Finetuned LLM attains 76.0%, outperforming the next-best baseline (Bi-LSTM, 61.2%) by 14.8 percentage points and the Decision Tree (55.0%) by 21.0 points. Both Non-Finetuned LLM baselines score below the traditional Decision Tree — GPT-5 at 36.0% and Non-Finetuned Qwen at 39.5% — despite their much larger model capacity.

Table 6: Lifecycle LCS-F1 comparison across models at window size $W = 5$ seconds.

Model	Lifecycle LCS-F1 (%)
Decision Tree	55.0
Bi-LSTM	61.2
GPT-5 (Non-Finetuned)	36.0
Qwen3.5-122B-A10B (Non-Finetuned)	39.5
Qwen3.5-122B-A10B (Finetuned)	76.0

Three patterns emerge from this ranking. First, the five models separate into three distinct tiers: the Finetuned LLM alone at the top (76.0%), the two traditional-ML baselines in the middle (55.0–61.2%), and the two Non-Finetuned LLMs at the bottom (36.0–39.5%). The gaps between tiers far exceed the variation within them: 6.2 points separate the two traditional-ML baselines and 3.5 points separate the two Non-Finetuned LLMs, whereas the Finetuned LLM leads the nearest baseline by 14.8 points. This tier structure suggests that the advantage stems not from model architecture alone but from specific design decisions in the proposed framework. Second, GPT-5 scores 3.5 points below the smaller Non-Finetuned Qwen backbone, indicating that larger model scale without domain adaptation does not improve lifecycle reconstruction.

6.4 Factor Contribution

The headline gain reported in Section 6.3 is attributable to three design decisions operating at three layers: the quality of the input passed to Stage 2 (the Graph-based Log Extraction, Section 6.4.1), the capability of the Stage 2 model itself (the fine-tuning regimen, Section 6.4.2), and the inference-time configuration (the time-window size, Section 6.4.3). Each subsection isolates one factor.

6.4.1 Graph-based Log Extraction: Clean Input vs. Noisy Input

This subsection evaluates Stage 1 from two complementary angles. Part 1 quantifies how clean the extracted logs $\hat{L}$ are intrinsically. Part 2 then measures the performance degradation when this cleanliness is compromised.

Part 1: Stage 1 produces a clean log substrate. Stage 1’s first contribution is volume reduction. Figure 7 reports the average per-lifecycle log and token counts before and after extraction across all 237 scenarios. The mean log count drops from 58,284 to 124.5 per lifecycle, a reduction of approximately two orders of magnitude; the corresponding token count drops from 57.99 million to 166,203, an equivalent reduction. The extracted volume fits within standard LLM context windows, whereas the original does not.

Volume reduction alone is useful only if the attack-relevant logs are preserved. Figure 8 reports the F1 distribution of Stage 1 across the 237 scenarios. The aggregated extraction achieves Recall 0.922, Precision 0.880, and F1 0.901, with a FPR of 2.57×10^{-4} . Per-lifecycle F1 values cluster tightly above 0.85 (median 0.900, 75th percentile 0.88), indicating consistent quality rather than a few high-performing outliers.

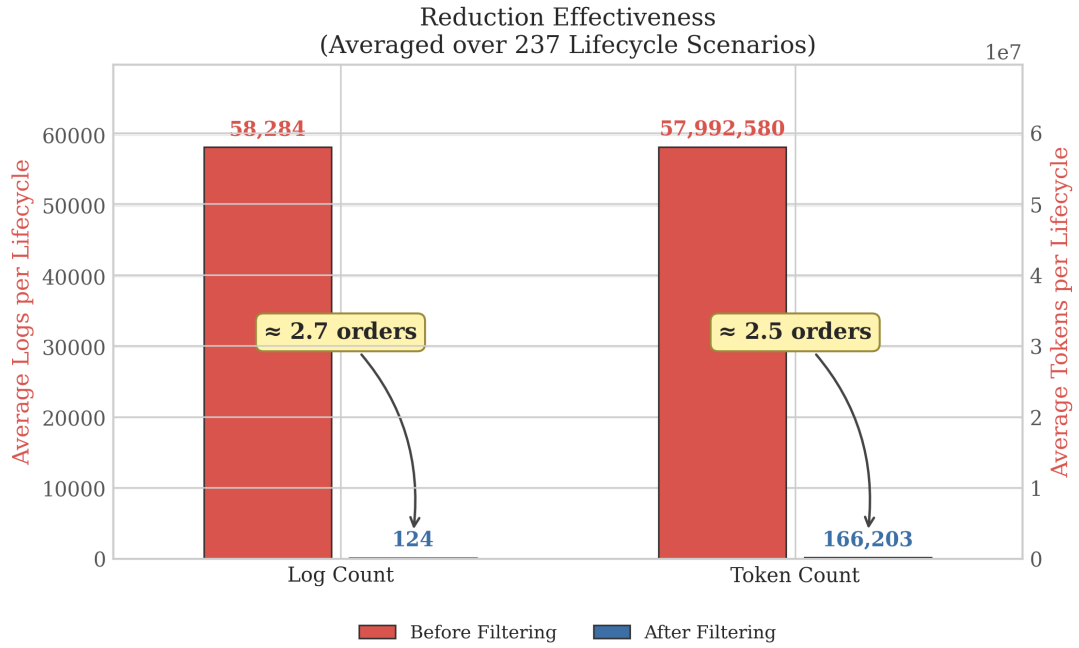


Figure 7: Per-lifecycle log and token counts before and after Stage 1 extraction, averaged over 237 lifecycle scenarios. Stage 1 reduces both quantities by approximately two orders of magnitude.

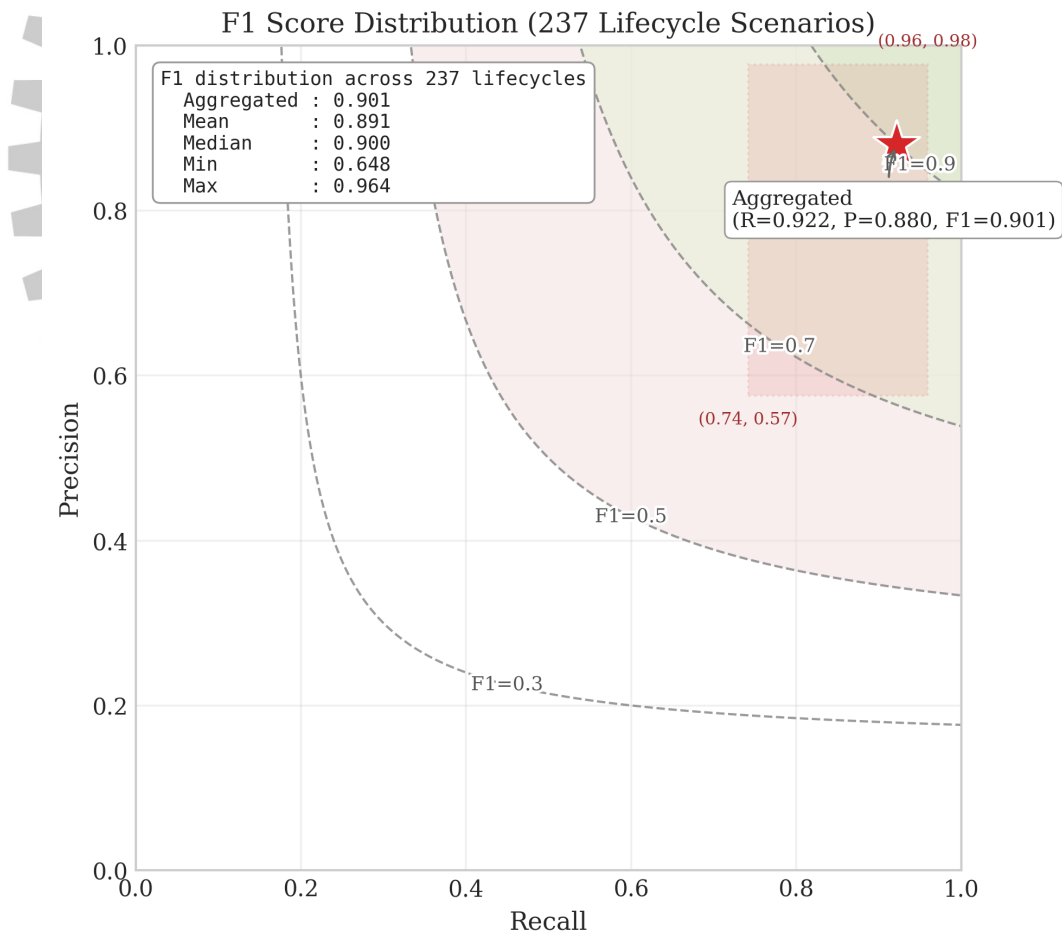


Figure 8: F1 distribution of Stage 1 extraction across 237 lifecycles. The aggregated point (red star) reports Recall = 0.922, Precision = 0.880, F1 = 0.901.

Part 2: downstream collapse without Stage 1. To quantify Stage 1’s contribution to the headline, the inputs to Stage 2 are progressively contaminated by injecting benign logs at ratios from 0% to 99%, simulating varying degrees of Stage 1 degradation. Noise injection serves as a proxy because a direct ablation is infeasible: raw log volume exceeds every model’s context budget. Figure 9 reports the resulting Lifecycle LCS-F1 across all five models.

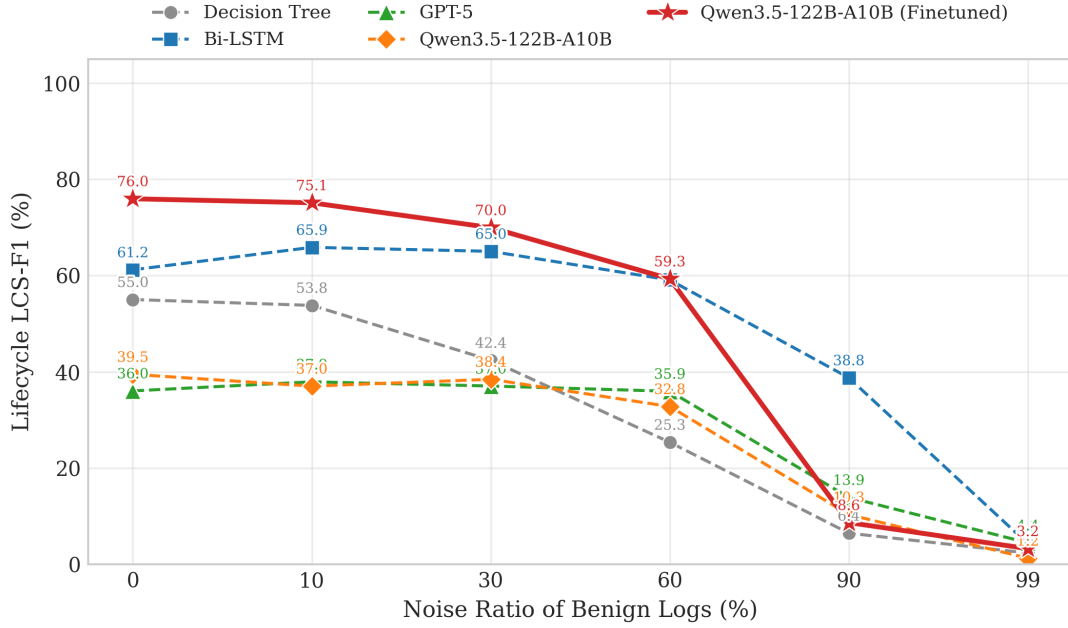


Figure 9: Lifecycle LCS-F1 versus benign-log noise ratio at $W = 5$. All models, including the Fine-tuned LLM, lose most of their performance once benign noise exceeds 60%.

The Finetuned LLM retains 75.1% LCS-F1 at 10% noise and 70.0% at 30%, but beyond 60% the curve falls sharply: 59.3% at 60%, 8.6% at 90%, and 3.2% at 99%. All other models follow the same pattern with lower absolute scores. Since raw system logs contain well over 99% benign entries (Section 6.1), operating without Stage 1 would place every model at or beyond the 99% noise level, reducing the Finetuned LLM from 76.0% to approximately 3%. Stage 1 is therefore not an optional preprocessing step but a structural prerequisite for the entire downstream pipeline.

6.4.2 LLM Fine-tuning: Finetuned vs. Non-Finetuned

This subsection isolates the contribution of fine-tuning while holding all other factors fixed. The Finetuned model and the Non-Finetuned model share the same backbone (Qwen3.5-122B-A10B), receive the same Stage 1 extracted logs $\hat{L}$, are queried with the same prompt format, and use the same time-window size $W = 5$. The only difference is whether the LoRA adapter (Section 5.4.1) is applied. Table 7 reports the result, with the Non-Finetuned GPT-5 included as a reference for a more capable but still Non-Finetuned closed-source alternative.

Table 7: Single-variable ablation: effect of fine-tuning under identical pipeline conditions ($W = 5$, same $\hat{L}$, same prompt).

Configuration	Lifecycle LCS-F1 (%)
Qwen3.5-122B-A10B (Non-Finetuned)	39.5
GPT-5 (Non-Finetuned, larger backbone)	36.0
Qwen3.5-122B-A10B (Finetuned)	76.0

Fine-tuning alone adds 36.5 percentage points relative to the Non-Finetuned version of the same backbone. Two observations sharpen this result. First, the Non-Finetuned GPT-5 — a model substantially larger than the 122B backbone used here — does not match the fine-tuned 122B model. The capability gap is therefore not explained by raw model scale but by domain-specific training. Second, the reasoning supervision protocol described in Section 5.4.1 requires the model to produce an explicit rationale that links log artifacts to ATT&CK technique definitions. This reasoning-augmented supervised fine-tuning teaches the model to ground its prediction in observed evidence rather than to rely on pre-training priors, which is consistent with the large fine-tuning gain observed here.

6.4.3 Time-Window Size: $W=5$ vs. Alternative Sizes

The window size W controls how Stage 2 partitions each alert-gap into windows before LLM classification (Section 4.3). To isolate the effect of this hyperparameter, W is swept over $\{2, 5, 15, 30, 60, 120\}$ seconds while all other components are held fixed. Figure 10 reports Lifecycle LCS-F1 and Figure 11 reports the corresponding Per alert-gap LCS-F1.

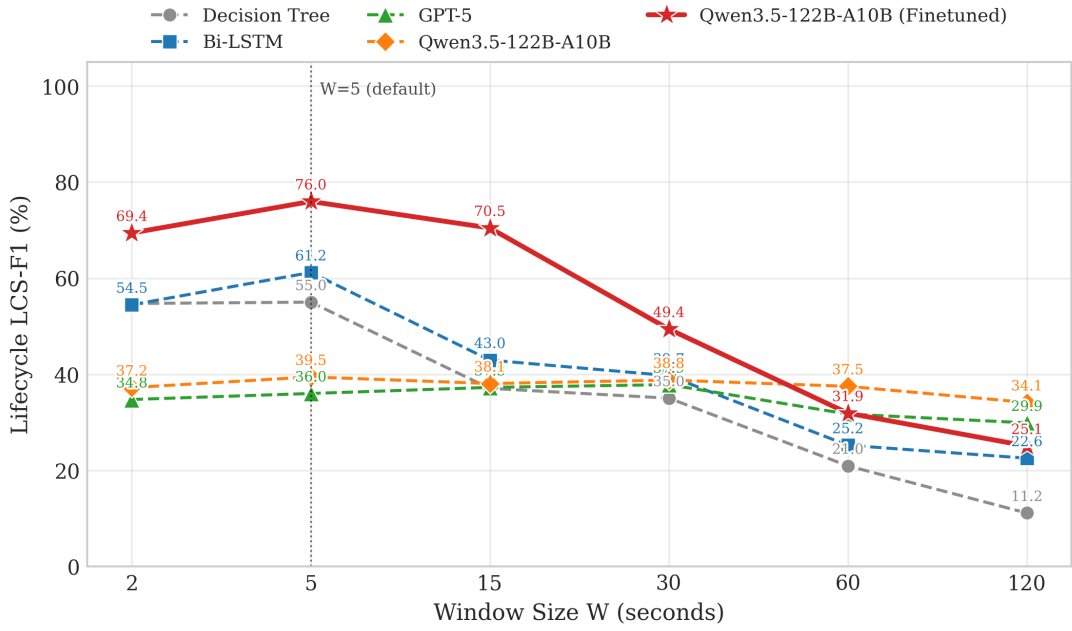


Figure 10: Lifecycle LCS-F1 as a function of window size W . The Finetuned LLM peaks sharply at $W = 5$.

The Finetuned LLM follows an inverted-U trajectory in both metrics. Lifecycle LCS-F1 rises from 69.4% at $W = 2$ to 76.0% at $W = 5$, then falls to 70.5% at $W = 15$, 49.4% at $W = 30$, 31.9% at $W = 60$, and 25.1% at $W = 120$. Per alert-gap LCS-F1 follows the same shape: 43.1% / 46.0% / 45.0% / 30.9% / 25.1% / 21.1% over the same window range. Two mechanisms explain the inverted-U shape. Below $W = 5$, individual technique executions are fragmented across multiple windows, so each window holds only a partial signature and the LLM cannot recover the full technique label. Above $W = 15$, windows begin to span multiple distinct techniques, violating the single-technique-per-window assumption of the classifier and

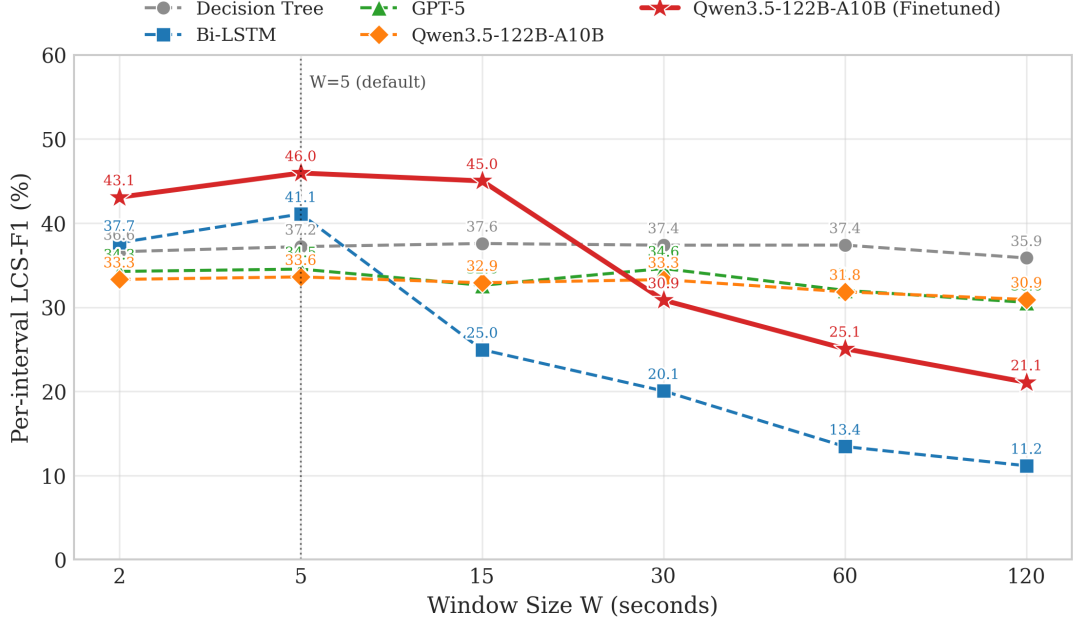


Figure 11: Per alert-gap LCS-F1 as a function of window size W . The Finetuned LLM and Bi-LSTM share the same peak location at $W = 5$.

causing the LLM to either conflate techniques or fall back to the most salient one. $W = 5$ minimizes both effects.

The Finetuned LLM exhibits a steeper decline at large W than the Non-Finetuned baselines: Qwen Non-Finetuned, for example, is comparatively flat at 37–40% across all W . This asymmetry has a natural interpretation: the fine-tuning training distribution centers on $W = 5$ windows, so inputs at substantially different windowing represent a distribution shift for the Finetuned model, whereas the Non-Finetuned baselines were never adapted to any particular W and therefore lack a strong W -specific optimum to lose.

6.5 Behavioral Analysis

The factor analysis in Section 6.4 identifies *which* design decisions contribute to the headline performance. This section turns to *under what conditions* that performance holds. Three complexity dimensions are examined, each corresponding to one processing unit of the pipeline:

the sample (Section 6.5.1), the alert-gap (Section 6.5.2), and the lifecycle (Section 6.5.3). Because these analyses observe model behavior rather than directly probe internal mechanisms, the discussion uses cautious phrasing where mechanism-level claims are suggested rather than proven.

6.5.1 Sample Complexity: Single-Technique vs. Multi-Technique Samples

At the sample level, per-window technique detection is the foundational classification task that feeds into lifecycle completion. Figure 12 reports the overall Micro-F1, Macro-F1, and LCS-F1 of per-window technique detection across the five models. The Finetuned LLM leads on all three metrics — 90.9%, 91.7%, and 94.0% respectively — with the next-best model (Bi-LSTM) trailing by 11–19 points.

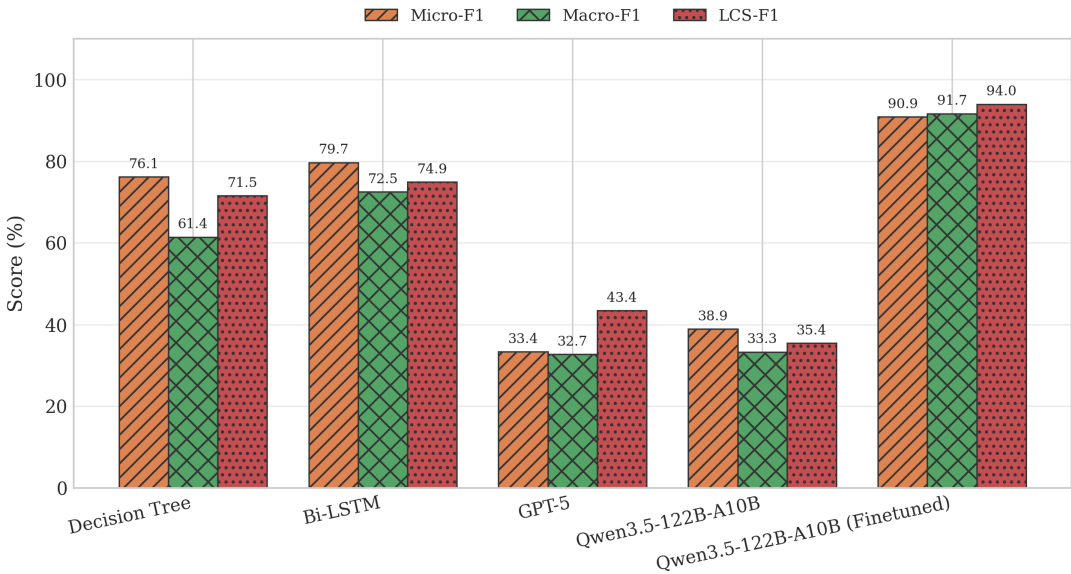


Figure 12: Overall per-window technique detection performance on the test split. The Finetuned LLM leads on Micro-F1, Macro-F1, and LCS-F1.

The overall numbers, however, average over samples of varying complexity. To examine where the lead is concentrated, the test split is stratified by the number of distinct techniques

present in each sample, $k \in \{1, 2, 3\}$. Figure 13 reports Micro-F1 per stratum.

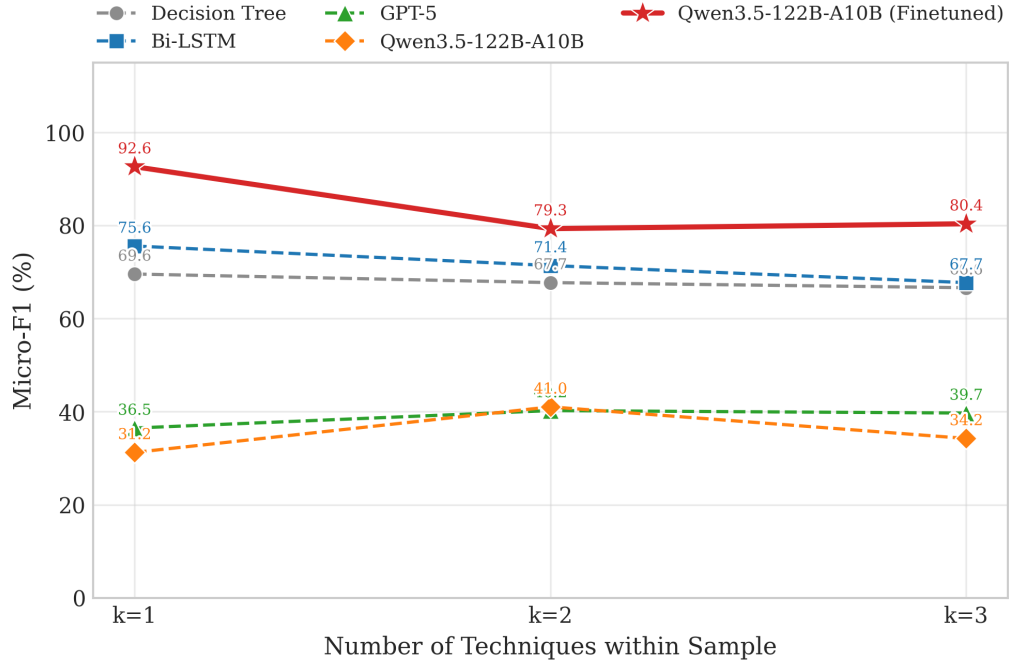


Figure 13: Per-window technique detection Micro-F1 stratified by the number of techniques per sample. Baselines degrade as k grows; the Finetuned LLM remains above 79% throughout.

The Finetuned LLM achieves 92.6% at $k = 1$, 79.3% at $k = 2$, and 80.4% at $k = 3$ — the only model holding Micro-F1 above 79% across the multi-technique range. The Decision Tree degrades from 69.6% to 66.6%, and the Bi-LSTM from 75.6% to 67.7%; both retain modest performance because the multi-label sigmoid head can in principle activate multiple outputs, but their feature representations are not designed to disentangle co-occurring techniques. The two Non-Finetuned LLMs collapse far below the traditional baselines: GPT-5 falls to 36–40%, and Qwen Non-Finetuned to 31–41%. The pattern is consistent with the hypothesis that the reasoning-augmented fine-tuning equips the Finetuned LLM with the ability to decompose composite samples into per-technique evidence chains, a capability that neither pre-training alone (the Non-Finetuned LLMs) nor surface-level pattern matching (the traditional baselines) supplies.

6.5.2 Alert-Gap Complexity: Sparse vs. Dense Alert-Gaps

At the alert-gap level, the framework processes each alert-gap between two alert-detected techniques as a unit (Section 4.3). The question this subsection addresses is whether alert-gaps containing many ground-truth techniques are systematically harder than sparse ones. Figure 14 reports Per alert-gap LCS-F1 stratified by the number of ground-truth techniques in the alert-gap.

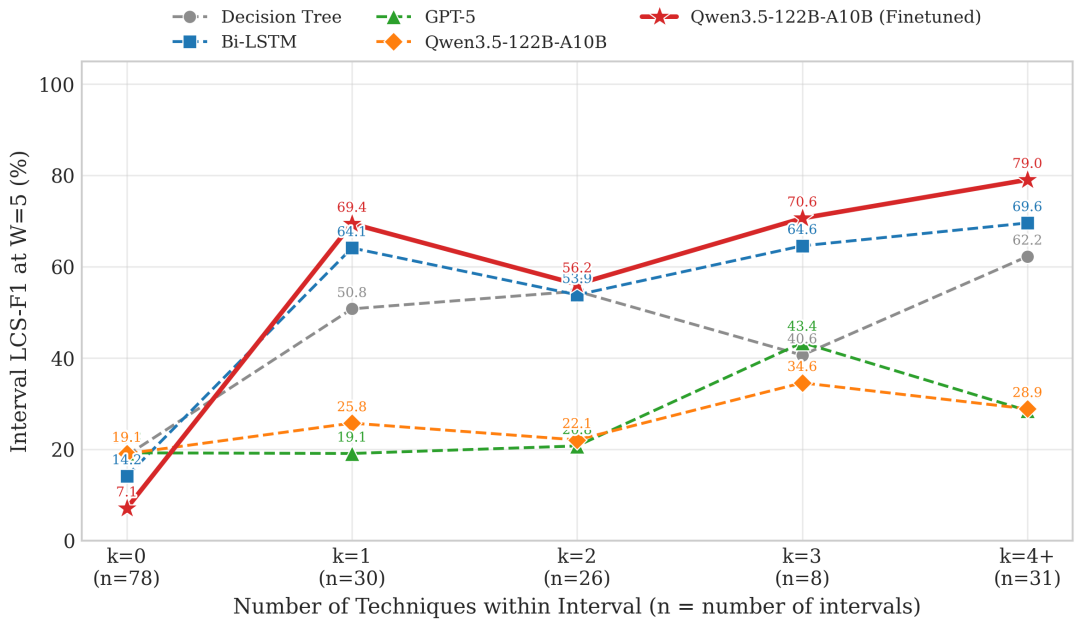


Figure 14: Per alert-gap LCS-F1 at $W = 5$ as a function of the number of ground-truth techniques per alert-gap. The Finetuned LLM does not degrade with denser alert-gaps.

For alert-gaps with at least one technique present ($k \geq 1$), the Finetuned LLM scores 69.4% ($k = 1, n = 30$), 56.2% ($k = 2, n = 26$), 70.6% ($k = 3, n = 8$), and 79.1% ($k = 4+, n = 31$). The denser strata are not systematically harder; the highest performance occurs at the densest stratum. The $k = 0$ stratum (78 alert-gaps labeled as benign) is excluded from the generalization claim because by construction any prediction over a benign alert-gap is a false positive, so the 7.1% score there reflects the cost of any prediction at all, not the model’s discrimination capacity.

This non-degradation is consistent with the design intent of the $W = 5$ segmentation. A dense alert-gap containing four techniques is decomposed into multiple short windows, each of which is dominated by at most one technique. The framework therefore converts a hard multi-technique-per-alert-gap problem into many easy single-technique-per-window problems, where the latter is precisely the regime in which the Finetuned LLM is strongest (Section 6.5.1). The modest dip at $k = 2$ (56.2%) is most likely attributable to the small sample size of that stratum ($n = 26$); the overall trend across $k = 1$ through $k = 4+$ shows no systematic degradation with density.

6.5.3 Lifecycle Scale: Short vs. Long Lifecycles

At the lifecycle level, the final question is whether longer lifecycles — which contain more alert-gaps and thus more LLM invocations — accumulate errors that degrade the final LC_C relative to LC_{CT} . Figure 15 reports Lifecycle LCS-F1 stratified by ground-truth lifecycle length.

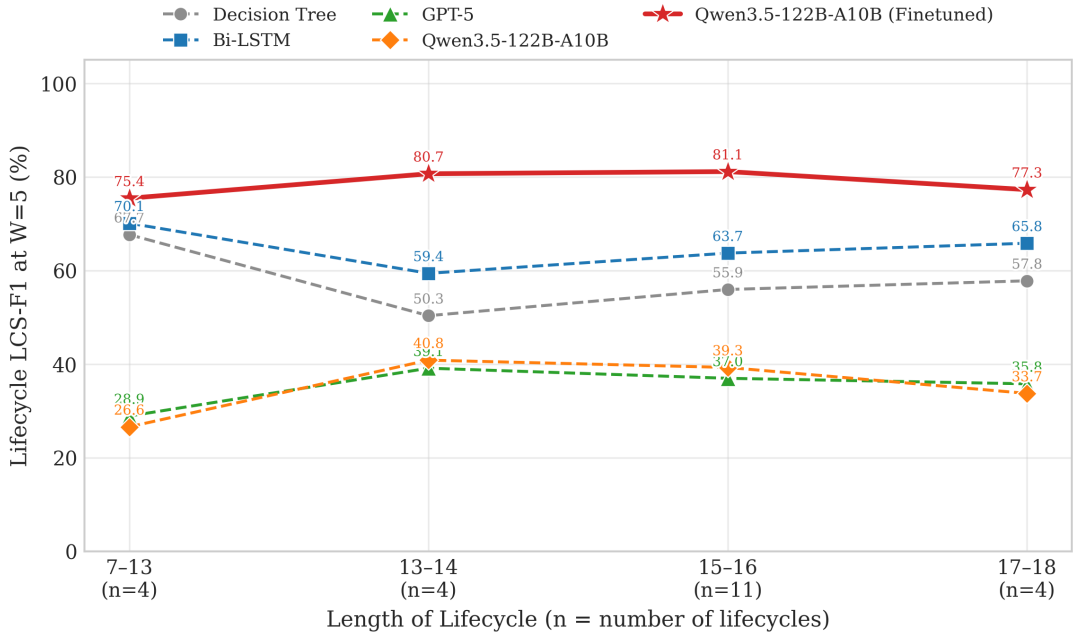
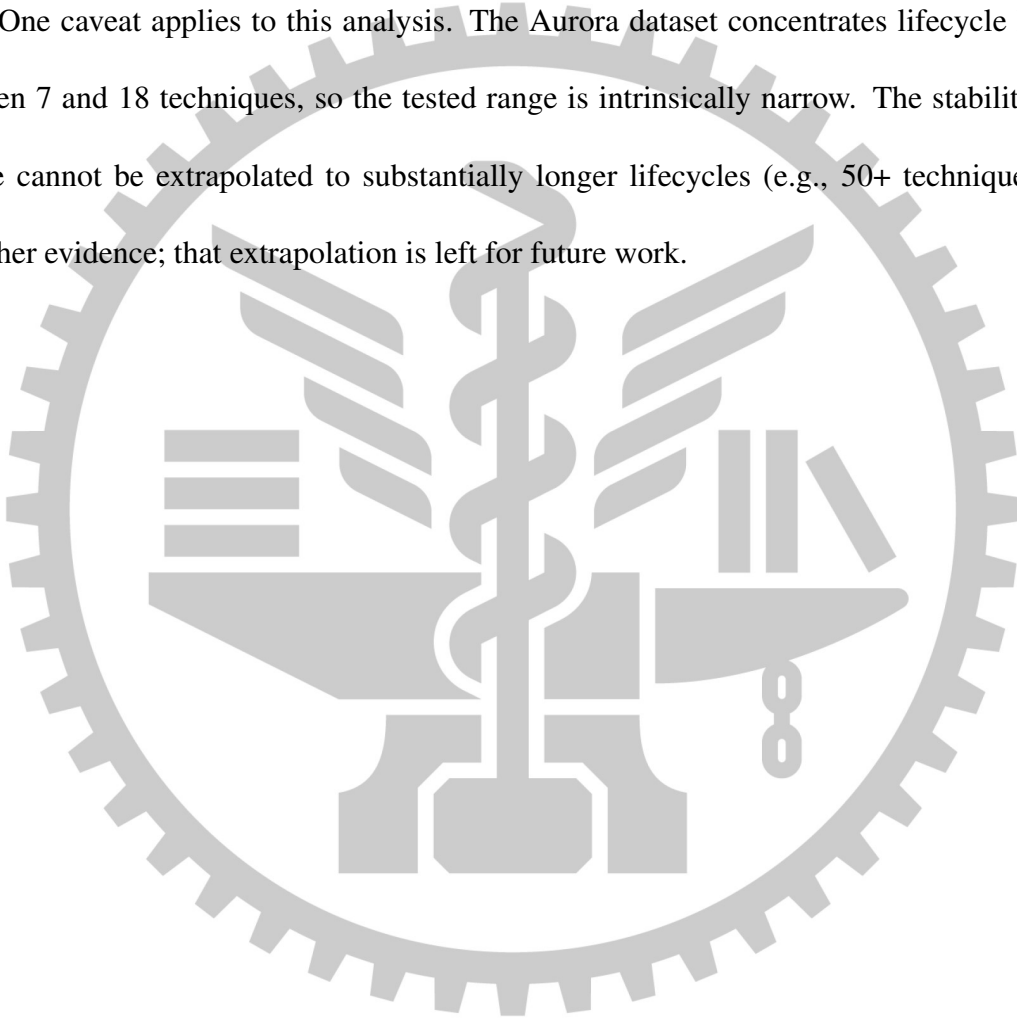


Figure 15: Lifecycle LCS-F1 at $W = 5$ stratified by ground-truth lifecycle length. The Finetuned LLM remains stable across the tested range.

The Finetuned LLM achieves 75.4% on length-7–13 lifecycles, 80.7% on length-13–14,

81.1% on length-15–16, and 77.3% on length-17–18. Performance does not degrade monotonically with length; the longest-bin score is comparable to the shortest. This stability is consistent with the framework’s alert-gap-based decomposition: because each alert-gap is processed independently and the final LC_C is assembled by concatenation rather than by a single end-to-end pass, the per-alert-gap prediction quality does not accumulate compounding errors as the lifecycle grows.

One caveat applies to this analysis. The Aurora dataset concentrates lifecycle lengths between 7 and 18 techniques, so the tested range is intrinsically narrow. The stability observed here cannot be extrapolated to substantially longer lifecycles (e.g., 50+ techniques) without further evidence; that extrapolation is left for future work.



Chapter 7.

Conclusion and Future Work

This thesis addresses the problem of completing an attack lifecycle from system and network logs when alerts cover only about 40% of the techniques an adversary executes. To close this gap, the thesis proposes a two-stage framework. Stage 1 (Graph-based Log Extraction) reduces raw multi-source logs to an attack-relevant subset via causal-graph construction and subgraph search. Stage 2 (LLM-Driven Lifecycle Completion) splits the extracted logs into time windows, classifies each with a fine-tuned LLM, and assembles the per-window predictions into the completed lifecycle.

Across 237 Aurora attack scenarios, the Finetuned LLM achieves a Lifecycle LCS-F1 of 76.0%, beating Bi-LSTM by 14.8 points and Non-Finetuned GPT-5 by 40.0 points. Stage 1 reduces the average log volume by roughly two orders of magnitude (58,284 to 124.5 entries per lifecycle) while retaining 92.2% of attack-relevant logs at F1 of 0.901. Without this extraction, the benign-log ratio exceeds 99% and the Finetuned LLM drops from 76.0% to about 3%, confirming Stage 1 as a structural prerequisite.

Three factor analyses isolate where the gain comes from. Fine-tuning the 122B Qwen backbone adds 36.5 points over its Non-Finetuned baseline — larger than the gap to GPT-5 — so the gain comes from domain-specific training, not model scale. The time-window $W = 5$ seconds sits at an inverted-U optimum: smaller windows split technique signatures, larger ones conflate multiple techniques. Behavioral analyses on the 23 test scenarios further show stable

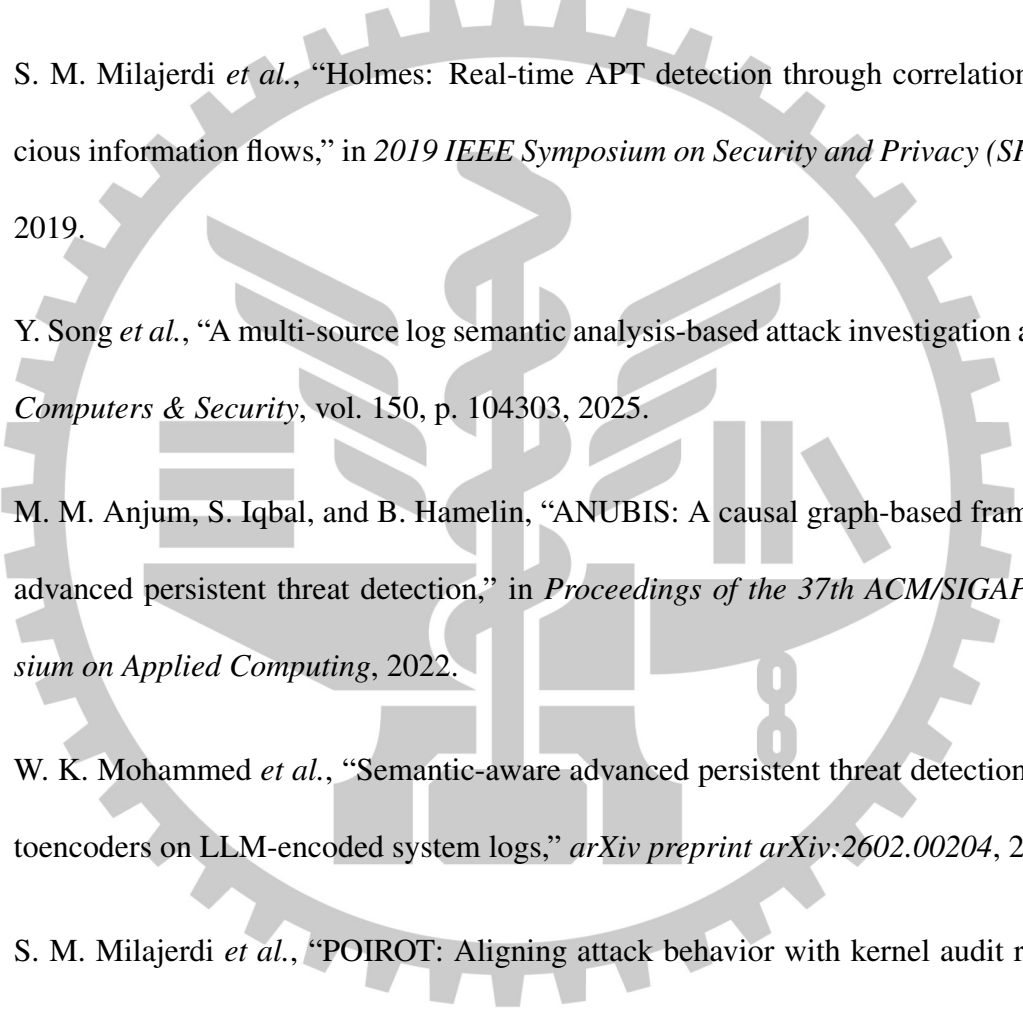
performance across single- and multi-technique samples, sparse and dense alert-gaps, and lifecycles of 7 to 18 techniques, suggesting that the framework’s effectiveness rests on problem decomposition rather than model capacity.

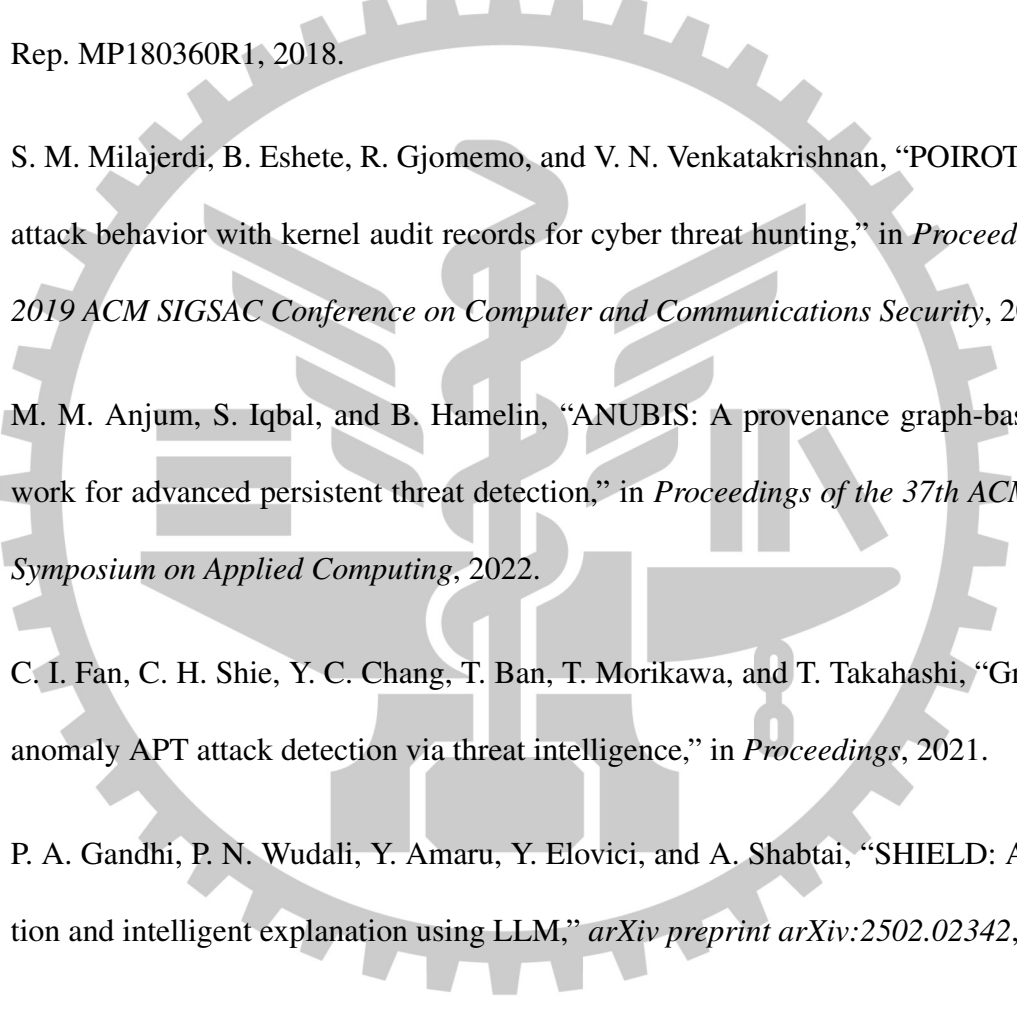
Four limitations bound these results. First, the Stage 1 contribution analysis uses noise injection as a proxy ablation, since raw log volume exceeds every model’s context window. Second, the behavioral analyses observe performance but do not probe model internals, so the proposed mechanisms — single-technique-per-window decomposition and gap-by-gap independence — remain inferred rather than directly shown. Third, test-split lifecycles span only 7 to 18 techniques; behavior on longer lifecycles in the full Aurora distribution (up to 37) and on real-world APTs is untested. Fourth, the reasoning rationales are not evaluated for faithfulness.

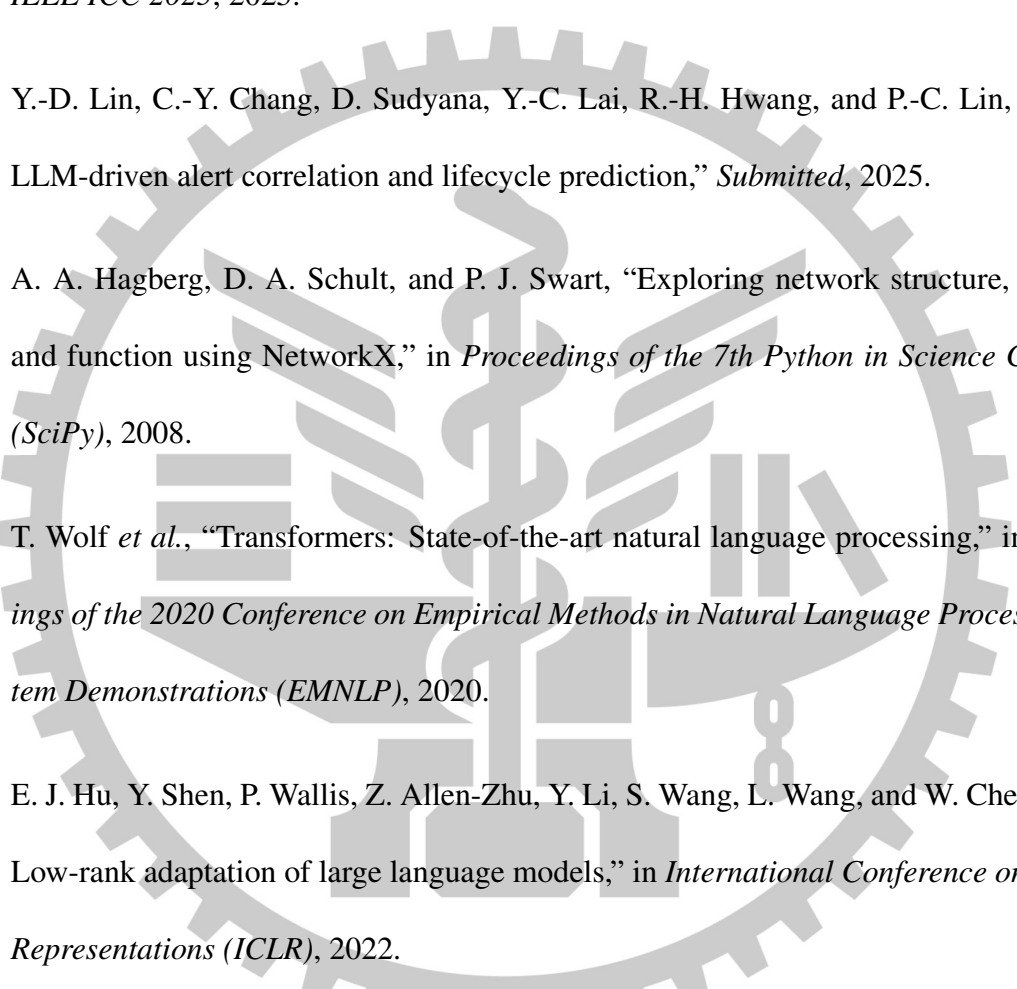
Several future directions follow. A held-out technique evaluation — using ATT&CK techniques unseen during training — would separate semantic learning from surface pattern matching. Per-technique error breakdowns, reasoning-trace faithfulness, and attention visualization would turn behavioral observations into mechanistic evidence. Evaluation on lifecycles of 20–30 or more techniques is needed before claiming length-invariance more broadly. Finally, a real-time deployment would require streaming Log2Graph, accelerated LLM inference, and SIEM/SOAR integration.

The two-stage decoupling of structural filtering from semantic reasoning is not specific to Aurora or to the Qwen backbone. The same principle — reduce noise structurally before applying an LLM for semantic classification — applies wherever raw evidence vastly outnumbers the signal, positioning this architecture as a building block for automated SOC pipelines.

References

- 
- [1] Y.-D. Lin *et al.*, “Two-stage multi-datasource machine learning for attack technique and lifecycle detection,” *Computers & Security*, vol. 142, p. 103859, 2024.
- [2] S. M. Milajerdi *et al.*, “Holmes: Real-time APT detection through correlation of suspicious information flows,” in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019.
- [3] Y. Song *et al.*, “A multi-source log semantic analysis-based attack investigation approach,” *Computers & Security*, vol. 150, p. 104303, 2025.
- [4] M. M. Anjum, S. Iqbal, and B. Hamelin, “ANUBIS: A causal graph-based framework for advanced persistent threat detection,” in *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*, 2022.
- [5] W. K. Mohammed *et al.*, “Semantic-aware advanced persistent threat detection using autoencoders on LLM-encoded system logs,” *arXiv preprint arXiv:2602.00204*, 2026.
- [6] S. M. Milajerdi *et al.*, “POIROT: Aligning attack behavior with kernel audit records for cyber threat hunting,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019.
- [7] P. A. Gandhi *et al.*, “SHIELD: APT detection and intelligent explanation using LLM,” *arXiv preprint arXiv:2502.02342*, 2025.

- 
- [8] J. Zhang *et al.*, “When LLMs meet cybersecurity: A systematic literature review,” *Cybersecurity*, vol. 8, no. 55, 2025.
- [9] S. Hans *et al.*, “Security logs to ATT&CK insights: Leveraging LLMs for high-level threat understanding and cognitive trait inference,” 2025.
- [10] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, “MITRE ATT&CK: Design and philosophy,” The MITRE Corporation, Tech. Rep. MP180360R1, 2018.
- [11] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. N. Venkatakrishnan, “POIROT: Aligning attack behavior with kernel audit records for cyber threat hunting,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019.
- [12] M. M. Anjum, S. Iqbal, and B. Hamelin, “ANUBIS: A provenance graph-based framework for advanced persistent threat detection,” in *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*, 2022.
- [13] C. I. Fan, C. H. Shie, Y. C. Chang, T. Ban, T. Morikawa, and T. Takahashi, “Graph-based anomaly APT attack detection via threat intelligence,” in *Proceedings*, 2021.
- [14] P. A. Gandhi, P. N. Wudali, Y. Amaru, Y. Elovici, and A. Shabtai, “SHIELD: APT detection and intelligent explanation using LLM,” *arXiv preprint arXiv:2502.02342*, 2025.
- [15] Y. Sharma, S. Birnbach, and I. Martinovic, “RADAR: A TTP-based extensible, explainable, and effective system for network traffic analysis and malware detection.” ACM, 2024.

- 
- [16] Y.-D. Lin, S. Y. Yang, D. Sudyana, F. Yudha, Y.-C. Lai, and R.-H. Hwang, “Two-stage multi-datasource machine learning for attack technique and lifecycle detection,” *Computers & Security*, 2024.
- [17] Y.-T. Huang, Y.-R. Guo, G.-W. Wong, and M. C. Chen, “A cascade approach for APT campaign attribution in system event logs: Technique hunting and subgraph matching,” in *IEEE ICC 2025*, 2025.
- [18] Y.-D. Lin, C.-Y. Chang, D. Sudyana, Y.-C. Lai, R.-H. Hwang, and P.-C. Lin, “LANCE: LLM-driven alert correlation and lifecycle prediction,” *Submitted*, 2025.
- [19] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in *Proceedings of the 7th Python in Science Conference (SciPy)*, 2008.
- [20] T. Wolf *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020.
- [21] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [22] L. Wang, Z. Li, Y. Jiang, Z. Wang, Z. Guo, J. Wang, Y. Wei, X. Shen, W. Ruan, and Y. Chen, “From sands to mansions: Towards automated cyberattack emulation with classical planning and large language models,” *arXiv preprint arXiv:2407.16928*, 2024.